

WEB3 工程师学习指南

Solidity 开发

类型系统、Storage、汇编、Foundry

Web3 Engineer Guide

版本 V1.0 · 2026.05

供个人学习使用 · MIT License

| 目录

01	章节路径图	.
02	第 1 章 类型系统：从二进制讲起	.
03	第 2 章 内存模型与 ABI 编码	.
04	第 3 章 函数：可见性、可变性、修饰符	.
05	第 4 章 继承与 C3 线性化	.
06	第 5 章 错误处理：require / revert / error / panic	.
07	第 6 章 事件与日志	.
08	第 7 章 assembly / Yul 深度章	.
09	第 8 章 库与生态：OpenZeppelin / Solady / Solmate	.
10	第 9 章 设计模式	.
11	第 10 章 Foundry 测试体系	.
12	第 11 章 部署与运维	.
13	第 12 章 Gas 优化清单	.
14	第 13 章 框架选型与备选语言生态	.
15	第 14 章 AI 辅助开发：实际生产力增益	.
16	第 15 章 实战项目	.
17	第 16 章 习题与参考解答	.
18	第 17 章 延伸阅读	.

2022 年 10 月, Audius 的 governance proxy 被升级。新实现合约「无意」地把一个 `bool` 字段加在了状态变量列表前面, 老的 `votingPeriod` slot 因此被解释成 0 ——治理脚本顺着这个 0 投了一票, 攻击者拿走 ~\$6M。三个月前, 一家审计公司用同样的姿势把 PoolTogether 的 v3 代理拍死过。

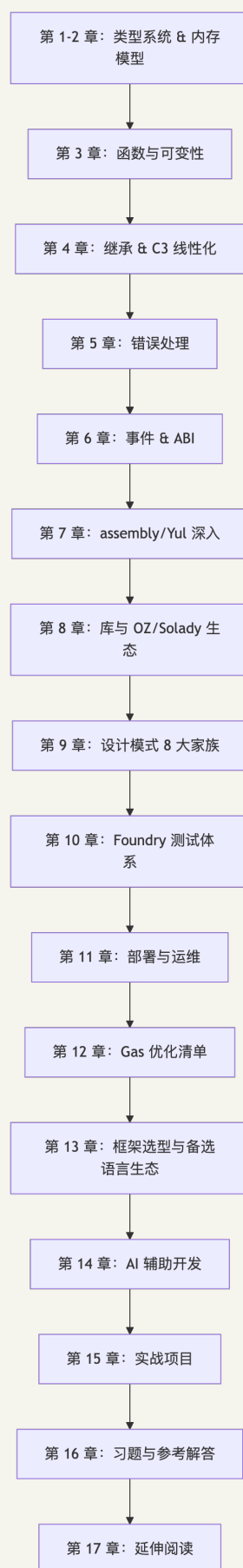
为什么同一个错误反复发生? 因为 Solidity 把状态变量按声明顺序铺到 32 字节的 storage slot——编译器不记你的字段名字, 只数字节。写 Solidity 的人, 必须能把任何 `.sol` 文件翻译成 slot 表。

这正是本模块的目的。前置是模块 03(EVM 执行模型、storage 布局、gas 计量)。学完之后, 你应该能: 读 Uniswap V4 / Aave V4 / Morpho / Lido 的源码不卡壳; 为它们写补丁和不变量测试; 对每一行代码说出它编译成什么 opcode、写到哪个 slot、为什么不能换序。

版本基线: Solidity 0.8.28(EVM target = cancun)、forge-std v1.9.5、OpenZeppelin Contracts v5.5.0、Solady v0.1.26。汇编 `TSTORE/TLOAD` 在 0.8.24+ 可用, `transient` 高层语法要等 0.8.29+(仅值类型, 1.7 节示例独立标注 0.8.29)。提醒一句: 0.8.28-0.8.33 的 `via_ir = true` 有清空高危 bug (2026-02-11 披露), 本仓库默认 `via_ir = false`, 生产建议升 0.8.34+。别踩这颗雷。

CHAPTER 01

章节路径图



四条验收线——读完这本模块之后，下面四件事你应该都能闭着眼做：① 拿到任何合约，都能在脑子里画出它的 storage layout 和 ABI；② 看到攻击场景，能立刻归类（重入 / 溢出 / 经济攻击 / 升级风险 / 签名重放 / front-run），并说出怎么修；③ 老板让你「砍 30% gas」，你知道该动 storage 打包、SSTORE2、Solady 还是汇编；④ 你写出来的代码：NatSpec 完整、不变量测试到位、storage 文档化、升级路径没有歧义。

CHAPTER 02

第 1 章 类型系统: 从二进制讲起

把开篇那段事故重新读一遍, 这次盯着代码看:

```
// V1 实现
contract TokenV1 { uint256 totalSupply; mapping(address => uint256) balanceOf; }

// V2 「热心」开发者把新字段加在前面
contract TokenV2 { address newOwner; uint256 totalSupply; mapping(address => uint256)
balanceOf; }
```

V1 部署后, proxy 的 slot 0 装着 totalSupply。开发者升级到 V2, proxy 还是同一份 storage, slot 0 的字节没动过——但 V2 现在把这 32 字节解释成 newOwner (一个 address)。totalSupply 跑到了 slot 1, 原来 slot 1 的 balanceOf 表头则跳到了 slot 2。一连串多米诺: mapping 算 balances[user] 时用错 slot, 所有人余额"归零"; 某个用户 transfer 一次, 下一秒 totalSupply 凭空变成了一个奇怪的地址。

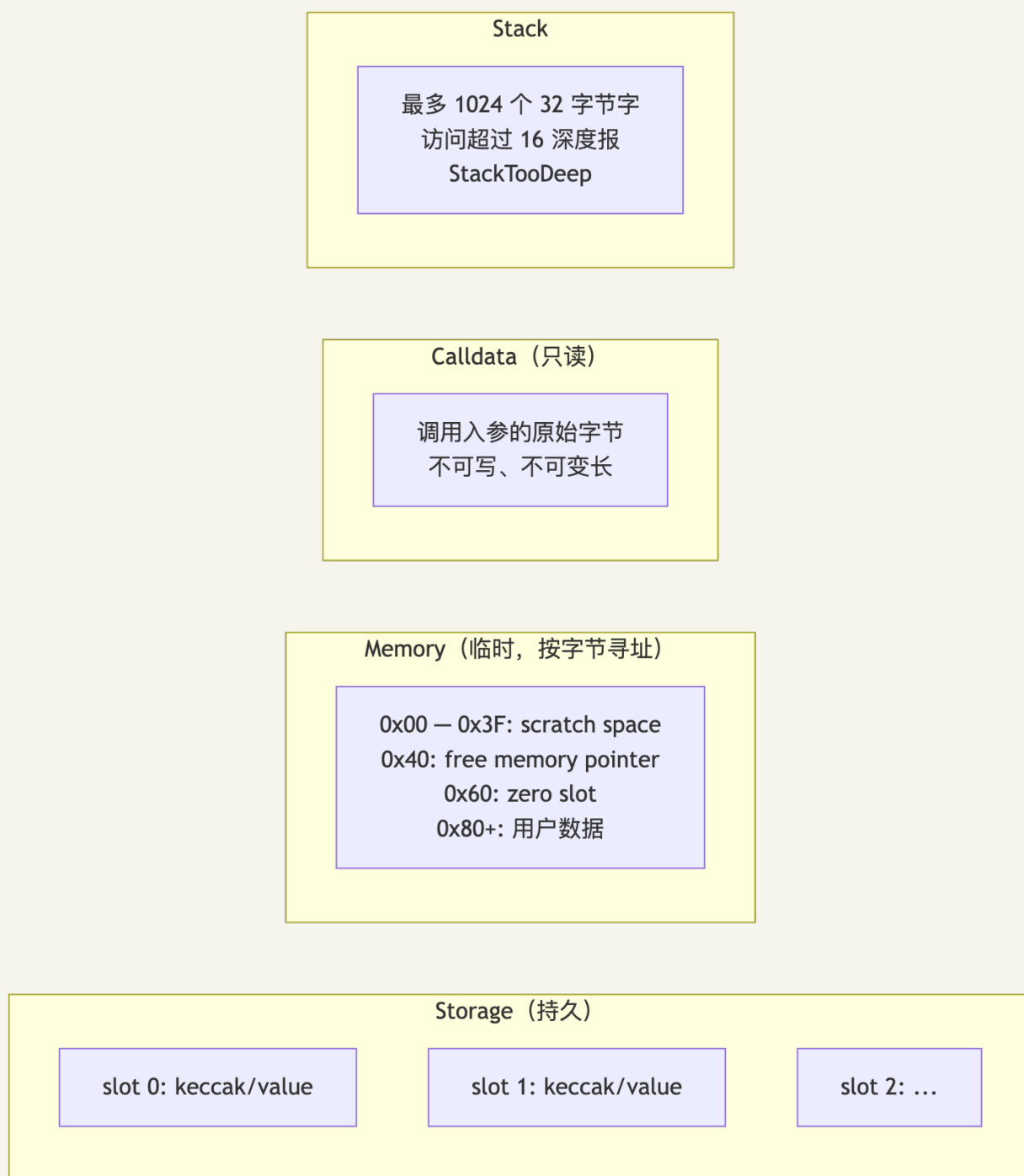
错的不是语法。语法层面 V2 完全合法。错在没人提醒这位开发者: Solidity 编译器看代码时只数字节, 不记字段名。状态变量是按声明顺序铺到 32 字节 slot 里的, 每加减、每换序、每改类型, 都是在重排你的硬盘扇区。

所以本章不教你「怎么声明变量」——那是 docs 第一页就有的事。本章教你把任何 .sol 文件翻译成 slot 表。语法只是壳, 真正决定运行时行为的是底层 layout。

1.1 为什么 storage slot 是 32 字节?

简单答案: 因为 Vitalik 当年抄了 keccak256 和 ECDSA 的作业。这两个原语都以 256 位为操作单位, EVM 选同样的字宽就不用拆位运算了。

代价你已经能猜到——存一个 bool 同样要烧 32 字节 SSTORE, 除非编译器能把它和邻居打包进同一个 slot。这件事我们 1.4 节会算账。先看四个数据区怎么分工。



记住这个类比，后面所有 gas 直觉都从这里来：

- **storage** 是合约的硬盘——贵、慢、持久。每次写一个新 slot 烧 22100 gas (差不多够一笔以太坊转账了)。改一个已经写过的 slot 也要 5000。读冷 slot 2100，热 slot 100。
- **memory** 是 RAM——临时、便宜、按字节寻址。但只增不减，函数内分配的内存一直占着，直到 tx 结束。
- **calldata** 是只读 ROM——最便宜、最快、连改都不让你改。external 函数的引用类型参数不写 calldata 是把油钱往火堆里扔。
- **stack** 是 CPU 寄存器——极快，但只能装 1024 个 32 字节的字，访问深度超过 16 直接报 StackTooDeep。

1.2 值类型完整列表

类型	字节数	说明	默认值	关键陷阱
<code>bool</code>	1	真假	<code>false</code>	不是 1 bit! 打包时占整字节
<code>int8 ~ int256</code>	1 ~ 32	有符号整数(步长 8)	0	0.8.x 默认 checked, 溢出 panic 0x11
<code>uint8 ~ uint256</code>	1 ~ 32	无符号整数	0	同上; <code>type(uint256).max</code> 是 $2^{256} - 1$
<code>address</code>	20	EOA 或合约地址	<code>0x0</code>	<code>address</code> 不能 <code>.transfer</code> ; 要 <code>address payable</code>
<code>address payable</code>	20	可收 ETH 地址	<code>0x0</code>	OZ v5 起 <code>Address.sendValue</code> 是更安全的接口
<code>bytes1 ~ bytes32</code>	1 ~ 32	定长字节数组	<code>0x00...</code>	与 <code>uint<M></code> 不同: <code>bytes</code> 是大端补零
<code>enum</code>	1(最多 256 项)	枚举	第 0 项	越界赋值 panic 0x21
<code>function</code> (内部)	8	函数指针	报 panic 0x51	<code>internal</code> 函数指针 = (合约地址, 函数 ID)
<code>function</code> (外部)	24	<code>address + selector(4)</code>	同上	<code>external</code> 函数变量打包: <code>address(20) + selector(4)</code>

陷阱深度框 #1: bool 不是 bit

每个新人都至少踩一次: 以为 `bool` 占 1 bit, 于是写一堆 `bool` 字段以为很省。事实是 `bool` 占 1 整字节 (`uint8` 的体面别名)。看这段:

```
contract BoolPacking {
    bool public a; // slot 0, offset 0, 占 1 字节
    bool public b; // slot 0, offset 1, 占 1 字节
    // slot 0 还剩 30 字节
    uint240 public c; // slot 0, offset 2, 占 30 字节, 刚好填满
    bool public d; // slot 1, offset 0, 新开一个 slot
}
```

真要 1 bit 的存储(比如几千人的白名单), 用 OZ 的 `Bitmaps` 或 Solady 的 `LibBitmap`。256 个布尔挤进 1 个 slot, 一次 SSTORE 翻盘。

1.3 引用类型与数据位置

下面这道题, 几乎每个新人都答错过。先猜: 执行 `bad()` 之后链上余额会变吗?

```

struct User { uint256 balance; uint256 lastSeen; }

mapping(address => User) public users;

function bad() external {
    User memory u = users[msg.sender];
    u.balance += 100;
}

```

答：不会。User memory u = users[...] 是复制——把两个 slot 读到 memory，新建一个独立副本。你在副本上 += 100，函数返回后副本被丢掉，链上 storage 一字未动。要真改链上，得用 User storage u = users[...]，那才是别名/指针。

引用类型必须显式声明 storage / memory / calldata：

```

function processOrders(
    Order[] calldata orders,           // 推荐：只读、不复制、最便宜
    uint256[] memory tempBuffer,       // 函数内临时使用
    Order storage firstOrder           // 从外部 mapping/数组借的引用
) external { /* ... */ }

```

三条规则记牢：

1. external 函数的引用类型参数 永远 用 calldata，除非你必须修改它（一笔 swap 路由的数组参数省 200~ 上千 gas，量大就显眼）；
2. internal 函数的引用类型参数用 memory 或 storage，看你是否要持久化修改；
3. storage 赋给 memory 是复制（烧 gas）；赋给 storage 是指针别名（几乎免费）。

完整对照：

```

function bad() external {
    User memory u = users[msg.sender]; // 复制：读两个 slot 到 memory
    u.balance += 100;                  // 改 memory，不改链上状态！
}

function good() external {
    User storage u = users[msg.sender]; // 别名：指针
    u.balance += 100;                  // 真改链上状态
}

```

1.4 storage layout 的二进制级推导

铺 slot 的规则其实只有四条：

1. 状态变量按声明顺序铺到 slot 0、1、2...
2. 每个变量从当前 slot 的下一个空字节开始放（左低位、右高位）
3. 如果当前 slot 剩余空间塞不下它，换新 slot（不会跨 slot 拼接）
4. 继承的父合约状态变量在子合约前面排（这就是为什么菱形继承可能炸）

immutable / constant 不占 slot——它们的值被嵌进 deployedBytecode (PUSH32 立即数)，不会出现在 forge inspect storage-layout 的 JSON 里。这是 1.6 节的伏笔。

下面这段代码考的就是上面四条规则——拿张纸自己先画一遍, 再看下面的"标准答案":

```
contract Layout {
    uint128 a; // slot 0, offset 0, 占 16 字节
    uint128 b; // slot 0, offset 16, 占 16 字节 → slot 0 满
    uint256 c; // slot 1 (256 位放不进剩余 0 字节)
    bool d; // slot 2, offset 0, 占 1 字节
    address e; // slot 2, offset 1, 占 20 字节
    uint64 f; // slot 2, offset 21, 占 8 字节 → 用了 29/32
    uint16 g; // slot 2, offset 29, 占 2 字节 → 用了 31/32
    uint8 h; // slot 2, offset 31, 占 1 字节 → slot 2 满 (32/32)
    uint256 i; // slot 3
}
```

forge inspect Layout storage-layout 输出:

```
{
  "storage": [
    {"label": "a", "offset": 0, "slot": "0", "type": "t_uint128"},
    {"label": "b", "offset": 16, "slot": "0", "type": "t_uint128"},
    {"label": "c", "offset": 0, "slot": "1", "type": "t_uint256"},
    {"label": "d", "offset": 0, "slot": "2", "type": "t_bool"},
    {"label": "e", "offset": 1, "slot": "2", "type": "t_address"},
    {"label": "f", "offset": 21, "slot": "2", "type": "t_uint64"},
    {"label": "g", "offset": 29, "slot": "2", "type": "t_uint16"},
    {"label": "h", "offset": 31, "slot": "2", "type": "t_uint8"},
    {"label": "i", "offset": 0, "slot": "3", "type": "t_uint256"}
  ]
}
```

4 个 slot——反例(每个变量一行随手写下来)会膨胀到 9 个 slot。算账: 每多一个 slot, 部署多花 ~20000 gas(首次 SSTORE), 运行时每读一次多一笔 SLOAD(冷读 2100, 热读 100)。一笔 swap 多读 5 个 slot 是 ~10000 gas, 挂着 100 万笔 swap 看, 存户分摊就是真金白银。

1.5 mapping、动态数组、字符串的位置算法

如果说定长字段是顺序排到货架上, mapping 就是用 keccak 当哈希索引——存哪儿全靠算。这个算法你必须能心算出来, 不然审计 mapping 状态、调试代理升级、写汇编访问都会卡。

mapping(K => V): 占一个 slot 但 slot 本身不存东西(永远是 0)。元素 `m[k]` 的位置是 `keccak256(abi.encode(k, slot))`。

```
contract Mapping {
    uint256 a; // slot 0
    mapping(address => uint256) balances; // slot 1 (占位)
    // balances[0xAAA] 的位置 = keccak256(abi.encode(0xAAA, 1))
}
```

`vm.load(address, keccak256(abi.encode(key, 1)))` 直接读链上 mapping 值——审计与状态恢复时有用。

嵌套 mapping: `m[k1][k2]` 在 `keccak256(abi.encode(k2, keccak256(abi.encode(k1, slot))))`, 每嵌套一层多算一次 keccak。

动态数组 `T[]` :

- `slot` 自身存 `length` ;
- 元素从 `keccak256(slot)` 开始连续铺, 每个元素按其类型大小占(小类型仍 packing)。

```
uint256[] arr; // slot 5
// arr[0] 在 keccak256(5), arr[1] 在 keccak256(5)+1, ...
```

`bytes` / `string` : 长度 ≤ 31 字节时打包进 `slot` 自身(最低字节存 `length*2`); 长度 > 31 字节时 `slot` 存 `length*2+1` , 数据从 `keccak256(slot)` 开始连续铺。

静态数组 `T[N]` : 连续铺 `N` 个 `T`, 仍 packing, 不存 `length`。

1.6 user-defined value types (UDVT, 0.8.8+)

零运行时开销的类型别名, 提供编译期类型安全:

```
type USD is uint256;
type WEI is uint256;
type Seconds is uint256;

function buy(USD amount, Seconds deadline) external { /* ... */ }

// 调用方必须显式 wrap, 编译器拒绝 uint256 ↔ USD 隐式转换
USD price = USD.wrap(100);
uint256 raw = USD.unwrap(price);

// 加自定义运算 (using for)
function add(USD a, USD b) pure returns (USD) {
    return USD.wrap(USD.unwrap(a) + USD.unwrap(b));
}
using {add as +} for USD global; // 0.8.19+: 操作符重载
```

适合: 单位类型(`USD/WEI/Seconds/Block`)、ID 类型(`UserId/PoolId/OrderId`)。ABI 里它退化为底层类型(`uint256`), 只在「编译期单位安全」收益大于「ABI 可读性损失」时使用。

1.7 Cancun 之后的 transient storage (必看)

Uniswap V4 团队第一次 demo `unlock(callback)` 模式时, 全场审计师集体懵——这种可重入风险拉满的写法, 怎么过得了 Trail of Bits? 答案是 EIP-1153: 他们用了一种新的存储层, 事务内有效、事务结束自动清零, 写一次 100 gas, 跟 warm SSTORE 持平。

这就是 transient storage。Cancun(2024-03)随它一起到的还有 EIP-5656(MCOPY)和大家盼了多年的 PUSH0。但 transient 是其中改动最深远的——它让 reentrancy lock 从 ~10000 gas 降到 ~200, 让 V4 的 hooks 架构成为可能。本节讲它的语法、四种生产模式、和那些教科书还来不及更新的坑。

数据区	持久性	gas(写)	gas(读)	对应 opcode
storage	永久	SSTORE 22100(冷)/ 5000(热)	SLOAD 2100/100	SSTORE/SLOAD
transient storage	当前 tx	100	100	TSTORE/TLOAD
memory	当前 call frame	3 + 二次方增长	3	MSTORE/ MLOAD
calldata	只读	-	2-3	CALLDATALOAD

Solidity 0.8.24 起支持以汇编访问 TSTORE/TLOAD; 0.8.29 起支持 高层语法(仅值类型):

以下高层 `transient` 语法需要 Solidity $\geq 0.8.29$ (仅限值类型); 其他章节示例使用本模块默认的 0.8.28, 需要 `transient` 时建议用 TSTORE/TLOAD 汇编版(见下文), 它在 0.8.24+ 全版本都能用。

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.29;

contract WithTransient {
    uint256 transient counter; // 0.8.29+ 高层语法 (值类型)

    function bump() external returns (uint256) {
        counter = counter + 1; // 编译为 TSTORE
        return counter;       // 编译为 TLOAD
    }
}
```

汇编版(兼容 0.8.24+, 含本模块默认的 0.8.28):

```
bytes32 constant SLOT = keccak256("MyContract.lock");

function _lock() internal {
    assembly { tstore(SLOT, 1) }
}
function _unlock() internal {
    assembly { tstore(SLOT, 0) }
}
function _isLocked() internal view returns (bool b) {
    assembly { b := tload(SLOT) }
}
```

经典用例: ReentrancyGuard 锁。OZ v5.2+ 的 `ReentrancyGuardTransient` 把每次调用开销从 ~5000 gas 降到 ~200。Uniswap V4 hooks 大量使用 `transient storage` 缓存 `swap` 中间状态。

踩坑: `transient` 不跨外部调用清零(同 tx 子调用仍能读到上层值), 所以「锁」可行, 「缓存」必须防污染。还有一个最容易被忘记的事——编译器不会自动清零 `transient` 变量。在你的函数末尾忘记写 0, 下一个 modifier 可能撞上脏数据。要么手动清, 要么彻底依赖 tx 结束(嵌套 call 时格外小心)。

1.7.1 transient storage 的四种生产模式

模式 A: reentrancy lock

```
modifier nonReentrant() {
    assembly { if tload(0) { revert(0, 0) } tstore(0, 1) }
    ~;
    assembly { tstore(0, 0) }
}
```

模式 B: 跨函数调用栈传 context (Uniswap V4 hooks 主用法)

V4 的 PoolManager 在 unlock(callback) 里把 LP 操作交给 callback 处理, callback 内部需要知道当前操作的池子, transient 避免把 poolId 逐层 thread 进每个调用:

```
contract PoolManager {
    bytes32 constant SLOT_LOCKER = keccak256("v4.locker");

    function unlock(bytes calldata data) external returns (bytes memory) {
        bytes32 slot = SLOT_LOCKER;
        assembly { tstore(slot, caller()) } // 记录谁解锁的
        bytes memory result = ICallback(msg.sender).callback(data);
        assembly { tstore(slot, 0) }
        return result;
    }

    function modifyLiquidity(PoolKey calldata key, ...) external {
        bytes32 slot = SLOT_LOCKER;
        address locker;
        assembly { locker := tload(slot) }
        require(msg.sender == locker, "must be inside unlock");
        // ... 真正的逻辑
    }
}
```

模式 C: 累积变量 (multicall 净结算)

multicall 调用多次 swap, 每次把 delta 累加到 transient slot, 结尾读一次净结算。比 storage 累加每次省 ~5000 gas。

```
function swap(int256 delta) external {
    bytes32 slot = keccak256(abi.encode("delta", msg.sender));
    assembly {
        let cur := tload(slot)
        tstore(slot, add(cur, delta))
    }
}

function settle() external {
    bytes32 slot = keccak256(abi.encode("delta", msg.sender));
    int256 net;
    assembly { net := tload(slot) tstore(slot, 0) }
    if (net > 0) IERC20(token).transferFrom(msg.sender, address(this), uint256(net));
    else if (net < 0) IERC20(token).transfer(msg.sender, uint256(-net));
}
```

模式 D: 批量授权 `cache`(Permit2 风格)

首次调用 Permit2 验证签名, 把 `user→spender→amount` 的临时授权写到 `transient`; 同 tx 内后续调用直接 `tload`, 免重新 ECDSA 验证。

安全规则:

- 1. 函数返回前手动清零(除非有意 `multicall` 累积)。嵌套 `call` 时 `keccak slot` 碰撞会相互污染, 不能依赖 tx 结束自动清零兜底。
- 2. `delegatecall` 共享 `transient`: `proxy` 实现合约的 `tstore` 实际写到 `proxy` 自己的 `transient`——`lib` 调用时要明确意图。
- 3. 没有 `forge inspect storage-layout` 等价工具: `transient layout` 全靠开发者维护, 审计时逐处 `review tload/tstore`。

1.7.2 EIP-5656 MCOPY: memory 复制指令

Cancun 的 `MCOPY` (EIP-5656)取代了逐 32 字节 `MLOAD + MSTORE` 的复制方式:

操作	Cancun 前	Cancun 后 (MCOPY)
复制 32 字节	<code>MLOAD(3) + MSTORE(3) = ~6 gas</code>	<code>MCOPY(3) + 3 = 6 gas</code> (持平)
复制 1 KB	<code>~10000 gas</code>	<code>~3300 gas</code>
复制 10 KB	<code>~100k gas</code>	<code>~33k gas</code> (省 80%)

Solidity 自动用 `MCOPY`: 0.8.24 仅支持 Yul 内联汇编; 0.8.25+ 编译器自动用于 `ABI encode/decode`、`bytes / string` 复制、数组复制。EVM target 设为 `cancun` 即生效, 不需要改代码。

显式 `memcpy`(仅在写极致优化的库时用):

```
function copyBytes(bytes memory src) internal pure returns (bytes memory dst) {
    dst = new bytes(src.length);
    assembly { memcpy(add(dst, 0x20), add(src, 0x20), mload(src)) }
}
```

验证生效: `forge inspect <Contract> assembly | grep -c MCOPY` , 不为 0 即已启用。

1.7.3 EIP-2930 access list(链下优化)

交易类型 `0x01`, 允许发送方预声明访问的 `address` 与 `storage slot`, 冷读 2100 gas 降到热读 100 gas(每槽省 ~2000)。

```
# cast 自动推导 access list 并构造 0x01 tx
cast send --access-list 0x... "swap(uint256)" 100 --rpc-url $RPC --private-key $PK
```

注意: `access list` 本身付费(每 `address` 2400、每 `slot` 1900), 跨合约调用至少 3 个槽才划算。Solidity 代码无需改动, 纯 tx 构造层优化。 `cast send --access-list` 会用 `eth_createAccessList` 自动推导 `access list`; `forge script --slow` 仅用于发送间隔等待, 不会自动加 `access list`。

CHAPTER 03

第 2 章 内存模型与 ABI 编码

2018 年的 Parity multisig hack 从一个细节开始：攻击者构造的 calldata 让初始化函数把 owner 写到了一个意想不到的位置。要看懂这种漏洞，要构造 delegatecall 的 calldata，要在汇编里跳过 ABI 解码读字段——你必须能心算每个字节的归宿。本章讲两件事：memory 怎么分区、ABI 编码到字节级的样子。

2.1 Memory 的区段

EVM memory 是按字节寻址的线性空间(非稀疏 keyed map)，编译器约定布局：

```
0x00 - 0x3F : scratch space (临时计算用, hash 输出等)
0x40 - 0x5F : free memory pointer (指向下一块空闲 memory)
0x60 - 0x7F : zero slot (永远是 0, 给空 bytes/string 用)
0x80 - ... : 用户数据 (动态分配)
```

编译器从 0x40 读指针，分配后往后挪。memory 只增不减，不能释放。汇编里手动写 memory 必须维护 0x40：

```
function alloc(uint256 size) internal pure returns (bytes memory ptr) {
    assembly {
        ptr := mload(0x40)           // 读当前 free pointer
        mstore(ptr, size)            // 在前 32 字节写 length
        mstore(0x40, add(ptr, add(size, 32))) // 推进 free pointer
    }
}
```

陷阱：汇编里直接 mstore(0x80, ...) 不更新 0x40，下一行 Solidity 分配 memory 时会从 0x40 指的位置(仍然是 0x80)开始覆盖写。你的“临时 buffer”转眼就成了别人新 string 的字节。这种 bug 找半天找不到，因为它是间歇性的——只在某条调用路径触发分配时才暴露。

2.2 ABI 编码：函数选择器与参数布局

函数调用在 EVM 层：

```
calldata = bytes4(keccak256("functionName(type1,type2,...)")) || abi.encode(arg1, arg2, ...)
           └────────── 4 字节 selector ───────────┘      └── 参数区 ───┘
```

函数选择器 = 函数签名(无空格、无返回值、无变量名、无 storage 位置)的 keccak256 前 4 字节：

```
transfer(address,uint256) → keccak256("transfer(address,uint256)")
                          → 0xa9059cbb...
                          → selector = 0xa9059cbb
```

参数编码规则(简化):

- 静态类型(`uint` , `address` , `bool` , `bytes32` , 静态数组): 直接 32 字节槽(左/右补零按规则)
- 动态类型(`bytes` , `string` , 动态数组、`T[]`):
- 在 "head" 位置存一个 32 字节的 `offset`(指向 "tail" 的位置)
- 在 "tail" 存 32 字节 `length`, 再存数据(按 32 字节对齐补零)

举例 `foo(uint256, bytes, address)` 调用 `foo(0x42, 0xCAFE, 0xAAAA)` :

```
offset 0x00: selector (4 bytes)
offset 0x04: 0x0000...0042      ← head: uint256
offset 0x24: 0x0000...0060      ← head: bytes 的 offset = 0x60
offset 0x44: 0x0000...AAAA      ← head: address
offset 0x64: 0x0000...0002      ← tail: bytes 的 length = 2
offset 0x84: 0xCAFE0000...      ← tail: bytes 数据 (左对齐补零到 32 字节)
```

理解此布局可: 手动构造 `delegatecall` / `call` 的 `calldata`; 在汇编里跳过 ABI 解码开销直接读字节; 调试 `vm.expectCall` / `cast 4byte-decode`。

Solidity 提供的 ABI 工具:

```
abi.encode(a, b)           // 标准 ABI 编码 (每个字段 32 字节对齐)
abi.encodePacked(a, b)     // 紧凑编码 (每字段按真实大小, 不补零) → 容易碰撞, 仅做 hash 用
abi.encodeWithSelector(ERC20.transfer.selector, to, amt)
abi.encodeCall(ERC20.transfer, (to, amt)) // 0.8.11+, 类型安全, 推荐
abi.decode(data, (uint256, bytes))        // 解码
```

陷阱(出过事故的真坑): `abi.encodePacked(a, b)` 在 `a`、`b` 都是动态类型时会哈希碰撞。

(`"ab"`, `"cd"`) 和 (`"a"`, `"bcd"`) 拼接出来的字节流完全一样——`"abcd"`。任何把它当签名输入的合约都会被这个等价签名重放。

铁律: 签名/`messageHash` 输入永远用 `abi.encode` (带长度前缀); `abi.encodePacked` 只在你能严格确认输入边界时用, 比如固定长度类型拼接。

2.3 EIP-712: 结构化签名

2024 年钱包钓鱼的主力武器之一: 诱骗用户签一个长得像普通 `transfer` 的 EIP-712 消息——实际上是 `Permit2` 的 `permitTransferFrom`, 签完瞬间被搬空。这种攻击成立的关键是 EIP-712 太通用了, 任何人都能定义自己的"消息类型", 钱包 UI 又往往只展示字段不展示语义。

但同样这把双刃剑也是 `Permit`、`Uniswap` 限价单、`Seaport` 订单簿、`CowSwap` MEV 保护、4337 `UserOperation` 的共同基础。学会它, 你拿到了 Web3 的离线协作语法。核心公式:

```
sign(keccak256("\x19\x01" || domainSeparator || hashStruct(message)))
```

其中:

```

// 1. domain separator (绑定 dapp、chain、合约)
bytes32 DOMAIN_TYPEHASH = keccak256(
    "EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"
);
bytes32 domainSeparator = keccak256(abi.encode(
    DOMAIN_TYPEHASH,
    keccak256(bytes(name)),
    keccak256(bytes(version)),
    block.chainid,
    address(this)
));

// 2. struct hash (绑定具体消息)
bytes32 PERMIT_TYPEHASH = keccak256(
    "Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadline)"
);
bytes32 structHash = keccak256(abi.encode(
    PERMIT_TYPEHASH, owner, spender, value, nonce, deadline
));

// 3. final digest
bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
address signer = ecrecover(digest, v, r, s);

```

为什么字段是这些？每一个都对应一种重放攻击：- `chainId` — 你在 `mainnet` 签的转账，没它就会在 `BSC` 上被重放(`chainId=1` vs `chainId=56`) - `verifyingContract` — 没它，A 合约的签名能被 B 合约接受，逻辑漏洞瞬间放大 - `nonce` (OZ `Nonces` 自增) — 没它，同一签名可被无限次提交 - `deadline` — 没它，半年前签的“小测试”在你忘记之后还活着

OZ v5 的 `EIP712.sol` + `ERC20Permit.sol` 已经把上面四件事都封好，本模块的 `MyToken.sol` 直接继承——你不需要自己拼 `hash`，但要看得懂它在拼什么。

CHAPTER 04

第 3 章 函数：可见性、可变性、修饰符

把这一章想成“函数签名上每个修饰词都在帮你或者坑你”。`public` 偷偷复制 `calldata` 进 `memory`，多花几百 `gas`；`view` 不是真的承诺“不写状态”——它只是编译期检查；`modifier` 里写 `return` 是个看似合法的语义炸弹。一行小词，账单上能差出三位数。

3.1 可见性(visibility)

关键字	谁能调	ABI 暴露	编译后实现
<code>public</code>	任何人(外部 + 内部)	是	既有 ABI 入口又有内部 dispatcher
<code>external</code>	仅外部 tx 或 <code>this.f()</code>	是	仅 ABI 入口；省掉内部 dispatcher
<code>internal</code>	本合约 + 子合约	否	等同于 C++ <code>protected</code>
<code>private</code>	仅本合约	否	子合约也不能调

优先 `external`：`public` 函数引用类型参数必须复制到 `memory`，`external` 直接读 `calldata`，省 200~上千 `gas`。`public` 状态变量的隐式 `getter` 是 `external-like` 的，但不能从子合约 `super.varName()` 访问(`getter` 是合成的，不是真函数)。

3.2 状态可变性(state mutability)

关键字	含义	链下可调？
默认	可读可写	是(要 tx 上链)
<code>view</code>	不写 <code>storage</code> ，可读	是(<code>staticcall</code> ，免 <code>gas</code>)
<code>pure</code>	既不读也不写 <code>storage</code>	同上
<code>payable</code>	可接收 ETH	同上

`view` 不是真保证不写——这一点你必须刻进 DNA。`view` 是编译期检查，不是运行时强制。一个用汇编 `sstore` 的合约可以挂着 `view` 标签照样写状态。所以审计别人的合约时，看到 `view` 不要松懈，要追每个外部调用看它真在做什么。`pure` 比 `view` 严格一档：连 `block.*`、`msg.*`、`tx.*`、其他合约状态都不让你读。

3.3 修饰符(modifier)

修饰符是代码注入，`_` 表示函数体展开位置：

```

modifier onlyOwner() {
    if (msg.sender  $\neq$  owner) revert NotOwner();
    -; // 函数体在这里展开
    // 后置代码 (很少用)
}

modifier nonReentrant() {
    if (_locked) revert Reentrant();
    _locked = true;
    -;
    _locked = false;
}

```

陷阱深度框 #2: `modifier` 里写 `return` 是反模式

```

modifier maybeSkip(bool skip) {
    if (skip) return; // 编译通过——但函数体根本不执行
    -;
}

```

读起来好像"跳过这次执行"。实际上：函数没跑，但也没 `revert`——所以返回值是该类型的零值。

`uint256` 返回 0，`bool` 返回 `false`，`address` 返回 `0x0`。调用方收到一个看起来"成功"的零值响应，状态啥也没变。下游逻辑依赖这个返回值时(比如"返回 `true` 才允许提现")，漏洞瞬间放大。

要中止流程，只有 `revert` 这一条路。`modifier` 里写状态变更也基本是反模式(重入锁、`Pausable` 这种基础设施除外)——OZ v5 的 `Ownable` / `AccessControl` / `Pausable` / `ReentrancyGuard` 都是审计过的，别在生产合约里重发明。

CHAPTER 05

第 4 章 继承与 C3 线性化

打开任何一个用 OZ 的项目，你都能看到这种诡异的继承列表：

```
contract MyToken is ERC20, ERC20Burnable, ERC20Permit, AccessControl { }
```

四个父类还看得出哪个干嘛。展开看：ERC20 自己继承了 Context, IERC20, IERC20Metadata, IERC20Errors，加上 ERC20Permit / ERC20Burnable / AccessControl 各自的链路，这是一个 8 层的菱形继承。

问题来了。当你 override `_update`，并且写 `super._update()`——它调的是谁？答错的代价：升级时 storage 字段错位（父类 layout 变了你没发现），虚函数被调到了老版本，资金按错的 fee 计算。

C3 线性化（和 Python 的 MRO 是同一个东西）就是答这个问题的算法。本章把它当作一道证明题做：先看几何图形，再走一遍合并步骤，最后告诉你为什么 OZ 文档要按“基类在前、扩展在后”的顺序写继承列表。

4.1 菱形继承与 C3 算法

多重继承用 C3 线性化（与 Python MRO 相同）决定 `super.foo()` 调谁。

菱形继承示例：



```
contract A { function foo() public virtual returns (string memory) { return "A"; } }
contract B is A { function foo() public virtual override returns (string memory) { return "B"; } }
contract C is A { function foo() public virtual override returns (string memory) { return "C"; } }
contract D is B, C {
    function foo() public override(B, C) returns (string memory) {
        return super.foo(); // 谁?
    }
}
```

C3 推导步骤：

```

L[A] = [A]
L[B] = [B, A]
L[C] = [C, A]
L[D] = [D] + merge(L[B], L[C], [B, C])
      = [D] + merge([B,A], [C,A], [B,C])

```

第 1 步：取 [B,A] 的 head = B。B 不在其他列表的 tail，选 B。

余下：merge([A], [C,A], [C])

第 2 步：取 [A] 的 head = A。A 在 [C,A] 的 tail，跳过。

取 [C,A] 的 head = C。C 不在其他列表的 tail，选 C。

余下：merge([A], [A], [])

第 3 步：取 [A]，A 不在任何 tail，选 A。

```
L[D] = [D, B, C, A]
```

所以 D.foo() 里 super.foo() 调的是 C 的 foo。is B, C 与 is C, B 顺序不同行为可能不同。OZ 文档明确建议继承顺序：

```

// OZ ERC20Burnable 文档建议：
contract MyToken is ERC20, ERC20Burnable, ERC20Permit, AccessControl { ... }
//                               ↑ 基类在前，扩展在后

```

4.2 override / virtual 规则

- 父函数要被覆盖必须 virtual (OZ v5 所有可覆盖函数都已标注)
- 子合约覆盖必须 override；菱形继承要 override(A, B, ...) 列出
- 可见性只能变宽(internal → public)，不能反向
- 可变性不能改 view / pure / payable 以外的组合

4.3 抽象合约与接口

```

abstract contract Base {
    function priceOf(address token) public view virtual returns (uint256); // 无函数体
    function someConcrete() external pure returns (uint256) { return 42; }
}

```

抽象合约有未实现函数，不能部署。接口无函数体、无状态变量、无构造器，只能有 function / event / error / type。与外部协议交互永远 import 接口而不是实现(OZ v5 提供 IERC20 / IERC721 / IERC4626 / IERC1271 等)。

4.4 storage gap 与升级安全

可升级合约的 storage 兼容性基于继承顺序，父合约 slot 永远在子合约前面。给父合约加新字段须预留 __gap：

```

contract BaseV1 {
    uint256 a;
    uint256 b;
    uint256[50] private __gap; // 预留 50 个 slot
}

contract BaseV2 {
    uint256 a;
    uint256 b;
    uint256 c; // 新加, 吃掉 gap[0]
    uint256[49] private __gap; // gap 缩成 49
}

```

OZ v5 起把状态变量搬到 ERC-7201 namespace storage, 不再依赖 **gap**。新合约推荐:

```

/// @custom:storage-location erc7201:myapp.token
struct TokenStorage {
    mapping(address => uint256) balances;
    uint256 totalSupply;
}

bytes32 private constant TOKEN_STORAGE = 0x...; // keccak256(...) - 1 & ~0xff

function _getStorage() private pure returns (TokenStorage storage $) {
    assembly { $ .slot := TOKEN_STORAGE }
}

```

namespace storage 完全隔离父子合约, 升级时无需担心父合约加字段破坏子合约布局。

CHAPTER 06

第 5 章 错误处理: require / revert / error / panic

回想一下你 MetaMask 里最近一次失败的 tx——那个让人摸不着头脑的红字弹窗。你看到的是 "ERC20: transfer amount exceeds balance" 还是 0xe450d38c 这种十六进制? 两者技术上是同一回事, 体验天差地别。

错误处理决定三件事: ① 用户在钱包里看到什么; ② 前端能否解码出原因; ③ 字节码占多少字节(贵的 bytecode 是开发者自掏的钱)。

`require(c, "ERC20: transfer amount exceeds balance")` 这串字符串会被编进 bytecode, 部署一次永久占着, 每次调用还要把它复制出来。OZ v5 把整个库切换到自定义错误, 省的就是这个钱——`InsufficientBalance(addr, bal, needed)` 编出来 4 字节 selector + 几个机器可读的字段, 前端按需翻译, 钱包按需展示。

5.1 三种 revert 方式

```
// 1. require (旧风格)
require(amount > 0, "amount must be positive"); // 字符串占字节码, ~200-400 gas

// 2. revert + 自定义错误 (推荐, 0.8.4+)
error InvalidAmount(uint256 provided);
if (amount == 0) revert InvalidAmount(amount); // 4 字节 selector, 省 ~30% runtime gas

// 3. assert (仅内部不变量)
assert(invariant); // 0.8.x 起只触发 Panic(0x01), 不再烧光 gas
```

自定义错误优势: 字节码更小(selector 4 字节 vs 字符串逐字节); 参数机器可读, 前端直接取值; 前端按需翻译 selector。

OZ v5 全面切换自定义错误(IERC20Errors、IERC721Errors 等):

```
error ERC20InsufficientBalance(address sender, uint256 balance, uint256 needed);
error ERC20InvalidApprover(address approver);
error AccessControlUnauthorizedAccount(address account, bytes32 role);
```

5.2 Panic codes 对照表

Panic 是编译器自动插入的检查失败:

代码	触发条件	修复
0x00	通用 panic	不应该发生
0x01	<code>assert(false)</code>	检查不变量逻辑
0x11	算术下/上溢(<code>unchecked</code> 块外)	用 <code>unchecked</code> 或换更大类型
0x12	除以 0 或对 0 取模	调用前检查
0x21	越界值赋给 enum	检查 enum 范围
0x22	错误编码的 storage byte 数组(不应手动写)	不要手动改 storage
0x31	<code>pop()</code> 空数组	调用前判空
0x32	数组越界	检查 length
0x41	分配过多 memory(>= ~1GB) 或长度溢出	限制循环或数组大小
0x51	调用未初始化的 <code>function</code> 类型变量	初始化或检查非空

5.3 try / catch (仅外部调用)

```

try otherContract.foo(arg) returns (uint256 r) {
    // 调用成功, r 是返回值
} catch Error(string memory reason) {
    // require/revert(...) 的字符串原因
} catch Panic(uint256 code) {
    // panic 编号
} catch (bytes memory data) {
    // 自定义错误或低层 revert, data 是 ABI 编码 (含 selector)
}

```

只能用于: 外部合约调用、`new Contract(...)` 创建、低层 `call/delegatecall/staticcall`。同一合约内部函数的 `revert` 直接冒泡, 不能 `catch`。

陷阱深度框 #3: catch 默认仍然消耗 gas

很多人以为 `try/catch` 是免费的"试错"。它不是。被调方按 EIP-150 拿走你剩余 gas 的 63/64, 即使最后 `revert`, `catch` 块拿到的也只剩可怜的零头。一个恶意被调方可以故意烧光 gas, 让你的 `catch` 后续逻辑跑不动。

```

try x.foo{gas: 50_000}() {} catch {} // 显式限 50k

```

`0Z Address.functionCall` 默认就建议显式限 gas。养成习惯: 调外部合约要么相信它要么限 gas, 没有"我先试一下不行就 `catch`"。

5.4 自定义错误进阶: 嵌套与 control flow

```

error InsufficientBalance(address account, uint256 needed, uint256 available);
error TransferFailed(address from, address to, bytes4 reason);

// reason 可以是另一个 error 的 selector
function withdraw(uint256 amt) external {
    if (balances[msg.sender] < amt) {
        revert InsufficientBalance(msg.sender, amt, balances[msg.sender]);
    }
    (bool ok, bytes memory ret) = msg.sender.call{value: amt}("");
    if (!ok) revert TransferFailed(address(this), msg.sender, bytes4(ret));
}

```

error 解码(链下):

```

// viem
const error = decodeErrorResult({
    abi: tokenAbi,
    data: '0xa1b2c3...' // 4 字节 selector + ABI 编码参数
});
// error = { errorName: "InsufficientBalance", args: [address, BigInt, BigInt] }

```

反模式: error as control flow: 每次外部调用 ~700 gas, 用 try-catch 做分支是把 if 写贵 100 倍。
error 是「不可恢复」状态, 控制流用 if/else。

OZ v5 的 IERC6093 标准化错误(ERC20/ERC721/ERC1155):

```

interface IERC20Errors {
    error ERC20InsufficientBalance(address sender, uint256 balance, uint256 needed);
    error ERC20InvalidSender(address sender);
    error ERC20InvalidReceiver(address receiver);
    error ERC20InsufficientAllowance(address spender, uint256 allowance, uint256 needed);
    error ERC20InvalidApprover(address approver);
    error ERC20InvalidSpender(address spender);
}

```

新合约直接 `import {IERC20Errors} from "@openzeppelin/contracts/interfaces/draft-IERC6093.sol"`
复用。Etherscan、viem、wagmi 自动识别并展示这些 error。

5.5 NatSpec 完整规范

所有 public/external 函数都应有 NatSpec——编译器把它编进 ABI metadata, 钱包签名 UI 直接展示。

5.5.1 标签全表

标签	用途	适用对象
@title	合约/接口标题	contract / interface
@author	作者署名	contract / interface
@notice	给最终用户看的描述(钱包 UI 会展示)	所有
@dev	给开发者看的实现细节	所有
@param	参数说明	function / event / error
@return	返回值说明	function(多返回值要逐个写)
@inheritdoc	从父合约继承文档	function(override 时极其有用)
@custom:xxx	自定义标签	所有(OZ 用 @custom:storage-location 标记 ERC-7201)

5.5.2 范例

```

/// @title 收益金库 v1
/// @author chuwuyo
/// @notice 把 USDC 存进来, 赚取 Aave 利息
/// @dev 继承 ERC4626, 重写 _withdraw 收 0.5% 提现费
/// @custom:security-contact security@example.com
contract YieldVault is ERC4626 {
    /// @notice 单次提现的费率, 单位 BPS (10000 = 100%)
    /// @dev immutable, 构造时确定
    uint256 public immutable FEE_BPS;

    /// @notice 当用户提现金额不足支付费用时抛出
    /// @param requested 请求提现的资产数量
    /// @param fee 应收的费用
    error InsufficientForFee(uint256 requested, uint256 fee);

    /// @notice 提现资产, 扣除费用后转给 receiver
    /// @param assets 提现的底层资产数量 (注意是 assets 不是 shares)
    /// @param receiver 收款人地址
    /// @param owner 份额持有者
    /// @return shares 实际烧毁的份额数量
    /// @inheritdoc ERC4626
    function withdraw(uint256 assets, address receiver, address owner)
        public
        override
        returns (uint256 shares)
    { /* ... */ }
}

```

5.5.3 自动生成文档: forge doc

```
forge doc --build          # 生成 docs/ 目录
forge doc --serve --port 4000 # 本地预览 (启动 mdbook)
```

forge doc 按合约/继承层级生成 mdBook 文档站点(OZ、Solady、Aave V3 的公开文档均用此生成)。CI: `forge doc --build && cd docs && mdbook build` 推 GitHub Pages。

5.6 SMTChecker: Solidity 内置形式化验证

编译器内置 SMT + Horn 求解器, 自动证明 require/assert 在所有输入下成立。免费零配置, 能找到 fuzz 漏掉的边界 bug。

5.6.1 启用方式

foundry.toml :

```
[profile.default]
model_checker = { engine = "all", timeout = 10000, targets = ["assert", "underflow", "overflow", "divByZero"] }
```

或者在合约里逐文件启用:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;

/// @custom:smtchecker { engine: "all", timeout: 10000 }
contract Counter {
    uint256 public count;
    function inc() external { count += 1; }
    function dec() external {
        assert(count > 0); // SMTChecker 会尝试证明这个永远成立或找到反例
        count -= 1;
    }
}
```

5.6.2 两个引擎

引擎	全称	擅长	限制
BMC	Bounded Model Checking	单函数内的所有路径、内联调用	不擅长循环(要给上限)
CHC	Constrained Horn Clauses	跨函数的状态不变量、循环	慢, 可能 timeout

`engine = "all"` 同时跑两者。

5.6.3 自动检查的目标

- `assert(...)` 是否永远成立
- 算术下/上溢(0.8.7+ 默认不查, 要显式打开)
- 除零、对零取模

- 数组越界
- ETH 余额不足
- 不可达代码

5.6.4 实战: 证明计数器永不下溢

```
pragma solidity 0.8.28;

contract Counter {
    uint256 public count;

    function dec() external {
        require(count > 0); // SMTChecker 把它当 assumption
        unchecked { count -= 1; }
        assert(count ≥ 0); // 自动证明 ✓
    }

    function unsafeDec() external {
        unchecked { count -= 1; } // 没 require
        assert(count ≥ 0); // SMTChecker 报错: count == 0 时减 1 下溢
    }
}
```

输出:

```
Warning: CHC: Assertion violation happens here.
Counterexample:
  count = 0
  call unsafeDec()
```

SMTChecker 是数学证明(非随机试值), 跨多函数调用序列、外部合约调用、复杂数据结构时会 timeout。

5.6.5 与 Halmos / Certora 的关系

工具	路径	何时用
SMTChecker	编译器内置, 免配置	写代码时随手开, 等同高级 lint
Halmos	a16z, 外部, Foundry 集成	复用 fuzz 测试当作 spec, 做 stateful 验证
Certora Prover	商业(学术免费)	高价值 DeFi 协议(Aave、Compound、Uniswap V4 都用)

推荐路径: CI 默认开 SMTChecker `targets = ["assert"]` → 关键不变量加 Halmos(章节 10.10) → 上 mainnet 大协议跑 Certora。

CHAPTER 07

第6章 事件与日志

把 event 当作“链下能看见、链上不能读”的廉价存档系统。一字节 SSTORE 要 5000+ gas，同样的字节用 LOG3 写到日志里不到 1500——价格相差 3 倍多。所有“用户看，合约不看”的数据都该走 event：转账历史、订单状态、参数变更、治理投票。这条经验法则贯穿本章。

6.1 EVM 日志层

LOG0 ~ LOG4 对应 0~4 个 topic，event 声明编译为 LOGN：

```
event Transfer(address indexed from, address indexed to, uint256 value);
//           topic1           topic2           data
//           topic0 = keccak256("Transfer(address,address,uint256)")
```

topic 数量限制：4 个。其中 topic0 永远是事件签名 hash，所以 indexed 字段最多 3 个。

gas：LOGN = 375 + 8 × bytes(data) + 375 × N，比 SSTORE(22100/5000)便宜一个数量级。「可链下索引但不需链上读取」的数据用 event 代替 storage。

6.2 indexed 字段的 hash 行为

```
event LogString(string indexed message);
emit LogString("hello"); // topic1 = keccak256("hello"), "hello" 原文不进 log
```

string / bytes / 动态数组作为 indexed 字段，topic 里存 keccak256(data)，原文不进日志。前端只能知道明文自行 hash 匹配，或增加一个非 indexed 字段保留原文。OZ ERC20 Transfer: addresses indexed, value 非 indexed(value 需链下聚合)。

6.3 anonymous 事件(少见但有用)

```
event Foo(uint256 a) anonymous;
```

匿名事件不带 topic0(无签名 hash)，最多 4 个 indexed 字段，但订阅者需用其他特征过滤。极少用。

CHAPTER 08

第 7 章 assembly / Yul 深度章

打开任何一个生产级 Solidity 项目的依赖目录：OZ 的 `MerkleProof._efficientHash`、Solady 的 `SafeTransferLib`、Uniswap V4 的 `transient lock`——全是 `assembly { }` 块。你写业务代码 99% 不会自己写汇编，但那 1% 的库决定了 99% 的 gas 账单和 100% 的安全边界。

读不懂这些汇编，你的审计报告永远止步于“调了某个库”。本章目的不是把你训练成 Yul 大师，而是让你能读、能审、必要时能改——并且当你真的不得不手写一段时，知道哪些线不能踩。

7.1 Yul 是什么

Yul 是 Solidity 的 IR 兼内联汇编语法，比 EVM opcode 多了局部变量（`let x := ...`）、函数、if/switch/for 控制流，但无类型系统——一切是 256 位字，一对一映射到 EVM opcode。

7.2 内联汇编基础

```
function add(uint256 a, uint256 b) external pure returns (uint256 c) {
  assembly {
    c := add(a, b)           // 把 a + b 赋给返回变量 c
    if iszero(c) { revert(0, 0) } // c == 0 时 revert
  }
}
```

规则：Solidity 变量在 `assembly` 里直接可访问（赋值用 `:=`）；赋值给命名返回变量即可，不需 `return`；`revert(p, s)` 从 `memory[p..p+s]` 读 revert data。

7.3 常用 opcode 速查

Yul 函数	对应 EVM opcode	用途
<code>add(a,b)</code> <code>sub(a,b)</code> <code>mul(a,b)</code> <code>div(a,b)</code>	ADD/SUB/MUL/DIV	算术(无溢出检查!)
<code>mod(a,b)</code> <code>addmod(a,b,N)</code> <code>mulmod(a,b,N)</code>	MOD/ADDMOD/MULMOD	模运算
<code>lt(a,b)</code> <code>gt(a,b)</code> <code>eq(a,b)</code> <code>iszero(a)</code>	LT/GT/EQ/ISZERO	比较(返回 0/1)
<code>and(a,b)</code> <code>or(a,b)</code> <code>xor(a,b)</code> <code>not(a)</code> <code>shl(s,a)</code> <code>shr(s,a)</code>	位运算	极致优化用
<code>mload(p)</code> <code>mstore(p,v)</code> <code>mstore8(p,v)</code>	MLOAD/MSTORE	memory 读写
<code>sload(p)</code> <code>sstore(p,v)</code>	SLOAD/SSTORE	storage 读写
<code>tload(p)</code> <code>tstore(p,v)</code>	TLOAD/TSTORE	transient storage (Cancun+)
<code>keccak256(p,s)</code>	KECCAK256	hash memory[p..p+s]
<code>call(g,a,v,ip,is,op,os)</code>	CALL	外部调用
<code>staticcall(g,a,ip,is,op,os)</code>	STATICCALL	只读调用
<code>delegatecall(g,a,ip,is,op,os)</code>	DELEGATECALL	代理执行
<code>return(p,s)</code> <code>revert(p,s)</code>	RETURN/REVERT	返回 / 回滚
<code>calldataload(p)</code> <code>calldatacopy(dp,sp,s)</code> <code>calldatasize()</code>	CALLDATA*	读 calldata
<code>caller()</code> <code>callvalue()</code> <code>chainid()</code>	CALLER/CALLVALUE/CHAINID	上下文

7.4 真实优化案例 1: 高效读 calldata 数组

ERC20 `batchTransfer` 的 Solidity 实现:

```
function batchTransfer(address[] calldata to, uint256[] calldata amt) external {
    require(to.length == amt.length, "len");
    for (uint256 i = 0; i < to.length; ++i) {
        _transfer(msg.sender, to[i], amt[i]);
    }
}
```

每次 `to[i]` 做边界检查 + calldata 解码。Solady 风格 Yul 版:

```

function batchTransfer(address[] calldata to, uint256[] calldata amt) external {
  assembly {
    // calldata 布局:
    // 0x04: to 的 offset (相对 calldata 起点 + 0x04)
    // 0x24: amt 的 offset
    // ... 各自的 length + 数据

    let toOffset := add(0x04, calldataload(0x04))
    let amtOffset := add(0x04, calldataload(0x24))
    let n := calldataload(toOffset)
    if iszero(eq(n, calldataload(amtOffset))) { revert(0, 0) }

    toOffset := add(toOffset, 0x20) // 跳过 length
    amtOffset := add(amtOffset, 0x20)

    for { let i := 0 } lt(i, n) { i := add(i, 1) } {
      let recipient := calldataload(add(toOffset, mul(i, 0x20)))
      let value := calldataload(add(amtOffset, mul(i, 0x20)))
      // ... 调用内部 _transfer 的汇编版
    }
  }
}

```

实测 50 个收款人省 ~25% gas。风险: 失去 checked 算术与编译器边界检查, 任何手抖都是 bug。

7.5 真实优化案例 2: 自定义 hash(OZ MerkleProof)

OZ 的 MerkleProof.processProof :

```

function processProofCalldata(bytes32[] calldata proof, bytes32 leaf) internal pure returns (bytes32) {
  bytes32 computedHash = leaf;
  for (uint256 i = 0; i < proof.length; i++) {
    computedHash = _hashPair(computedHash, proof[i]);
  }
  return computedHash;
}

function _hashPair(bytes32 a, bytes32 b) private pure returns (bytes32) {
  return a < b ? _efficientHash(a, b) : _efficientHash(b, a);
}

function _efficientHash(bytes32 a, bytes32 b) private pure returns (bytes32 value) {
  assembly {
    mstore(0x00, a)
    mstore(0x20, b)
    value := keccak256(0x00, 0x40)
  }
}

```

为什么用汇编: keccak256(abi.encode(a, b)) 复制到 memory 0x80+ 再 hash, 要分配 memory 并更新 free pointer; 汇编版直接用 0x00-0x3F scratch space, 省 200~ gas/次, 20 层 Merkle 树累计 4000+ gas。

7.6 真实优化案例 3: immutable

`immutable` 不存 storage, 编译器把值嵌入字节码(PUSH32), 零 SLOAD, 省 ~2100 gas(冷读)/ 100 gas(热读)。构造时确定、永不变的值都应该 `immutable`。

7.7 真实优化案例 4: transient storage 的可重入锁

OZ v5 `ReentrancyGuardTransient` (简化版):

```
abstract contract ReentrancyGuardTransient {
    // ERC-1967-like slot derivation
    bytes32 private constant REENTRANCY_GUARD_STORAGE =
        0x9b779b17422d0df92223018b32b4d1fa46e071723d6817e2486d003becc55f00;

    error ReentrancyGuardReentrantCall();

    modifier nonReentrant() {
        _nonReentrantBefore();
        _;
        _nonReentrantAfter();
    }

    function _nonReentrantBefore() private {
        if (_reentrancyGuardEntered()) revert ReentrancyGuardReentrantCall();
        assembly { tstore(REENTRANCY_GUARD_STORAGE, 1) }
    }

    function _nonReentrantAfter() private {
        assembly { tstore(REENTRANCY_GUARD_STORAGE, 0) }
    }

    function _reentrancyGuardEntered() internal view returns (bool entered) {
        assembly { entered := tload(REENTRANCY_GUARD_STORAGE) }
    }
}
```

对比传统 `ReentrancyGuard` (用 storage):

操作	storage 版	transient 版	节省
进入函数(写锁)	SSTORE 5000(热)	TSTORE 100	4900
退出函数(清锁)	SSTORE 5000(写 0 到非 0 退款)	TSTORE 100	4900
每次调用合计	~10000	~200	~9800

DEX 的每笔 swap 都 `nonReentrant`, 节省非常可观。

7.8 不要这么写: assembly 常见 footgun

```
// 1. 忘记更新 free memory pointer
assembly {
    mstore(0x80, value) // 写了 memory 但没动 0x40
}
// 下一行 Solidity 代码: bytes memory b = "..."; 会从 0x80 开始覆盖你写的!

// 2. 用 invalid() 而不是 revert(0, 0)
assembly {
    invalid() // 烧光所有 gas, 用户体验极差
}
// 应该写 revert(0, 0)

// 3. 在 storage slot 上手动 keccak 算偏移然后写错
bytes32 slot = keccak256(abi.encode(key, 1));
assembly { sstore(slot, value) }
// 如果 mapping 的 slot 不是 1 (继承改了顺序), 数据写到错误位置

// 4. 假设 calldata 布局
assembly {
    let x := calldataload(0x04)
}
// 如果你的函数是 (bytes calldata) 而不是 (uint256), 0x04 是 offset 不是 value
```

审计 + 测试是唯一安全网: 每个 assembly 块都要有专门测试覆盖正常路径 + 边界(length=0、超长、对齐错误)。手写汇编没有 checked 算术、没有边界检查、没有类型系统——所有传统编译器替你兜底的事, 全得自己写测试覆盖。

CHAPTER 09

第 8 章 库与生态: OpenZeppelin / Solady / Solmate

我面试过的 Web3 工程师里,能完整背出 ERC20 实现的不少。能告诉我"为什么生产合约里不该自己写 `transferFrom`"的,少得多。

答案是:你写的版本没经过 5 亿美金 TVL 验证、没被 Trail of Bits / OpenZeppelin / Spearbit 各审三遍、没修过 USDT 不返回 bool 的暗坑、没考虑 fee-on-transfer token 的差额、也没人告诉你 ERC-777 hook 会让你的"安全转账"暗中递归回来。

新合约的第一行不该是 `pragma`, 应该是 `import {ERC20} from "@openzeppelin/..."`。自己写 `transferFrom` 是面试题,不是生产代码。本章梳理三大库的定位差异,告诉你哪些应该混用、哪些坚决不能。

8.1 三大库定位

库	哲学	何时选
OpenZeppelin Contracts	严谨、审计过、保守	默认, 生产合约标准库
Solady	极致 gas 优化、激进汇编	性能敏感热路径
Solmate	早期 gas 优化范本	Yearn / Sound 等仍在用

实战: 主合约继承 OZ (ERC20/AccessControl/UUPS), 热路径用 Solady 的 `SafeTransferLib` / `FixedPointMathLib` / `LibBitmap`。两者同名实现不要混 `import` (命名空间冲突)。

8.2 OpenZeppelin v5.x 关键变化(相对 v4.x)

1. `Ownable()` 必须显式传 `Ownable(msg.sender)`
2. 所有 `require` 改自定义错误
3. 删除 `Counters` / `SafeMath` (0.8 内置 `checked`)
4. `AccessControl` admin role 必须显式 `grant`
5. `upgradeTo` 改名 `upgradeToAndCall`
6. 新增 ERC-4626、ERC-1271、ERC-7201 namespace storage

迁移: OZ 提供 `@openzeppelin/wizard` 自动重写大部分, 但 storage layout 兼容性须手工核对。

8.3 Solady 的几个杀手级工具

8.3.1 `SafeTransferLib`

OZ `SafeERC20` (简化):

```
function safeTransfer(ERC20 token, address to, uint256 value) internal {
    bytes memory data = abi.encodeCall(ERC20.transfer, (to, value));
    bytes memory returndata = address(token).functionCall(data);
    if (returndata.length > 0) {
        require(abi.decode(returndata, (bool)), "ERC20: transfer failed");
    }
}
```

Solady 版(Yul):

```
function safeTransfer(address token, address to, uint256 amount) internal {
    assembly {
        mstore(0x14, to)           // 0x10 - 0x33 存 to
        mstore(0x34, amount)       // 0x34 - 0x53 存 amount
        mstore(0x00, 0xa9059cbb000000000000000000000000) // selector + padding
        // call(gas, addr, value, in_ptr, in_size, out_ptr, out_size)
        if iszero(and(or(eq(mload(0x00), 1), iszero(returndatasize()))), call(gas(), token,
0, 0x10, 0x44, 0x00, 0x20))) {
            mstore(0x00, 0x90b8ec18) // TransferFailed selector
            revert(0x1c, 0x04)
        }
        mstore(0x34, 0) // 清掉 dirty memory, 给后续代码用
    }
}
```

省 ~30% gas, 可读性彻底归零。这就是 Solady 的哲学: 把"基础设施级"的库压到极致, 让所有应用合约共享性能红利。审计过的基础合约用 Solady, 业务层逻辑用 OZ SafeERC20——读得懂、改得动比省那一点 gas 更重要。

8.3.2 FixedPointMathLib

mulDiv/sqrt/ln/exp 高性能实现, 比 OZ Math.sol 快 2-5 倍, Compound/Aave 利率计算标配。

8.3.3 SSTORE2 / SSTORE3

数据存为合约字节码而非 storage, 写贵但读极便宜:

方式	写 1KB 成本	读 1KB 成本
SSTORE	~720,000 gas (22k × 32 slot)	~6,400 gas (200 × 32)
SSTORE2 (CREATE)	~210,000 gas (200 gas/byte)	~3,400 gas (EXTCODECOPY + 100/byte)

适用: 写一次读多次的大块数据(NFT metadata、配置 blob、merkle root 历史)。

API:

```
import {SSTORE2} from "solady/utils/SSTORE2.sol";

address pointer = SSTORE2.write(data); // 部署一个新合约, bytecode = data
bytes memory data2 = SSTORE2.read(pointer);
```

SSTORE3 用 CREATE2 + 固定 salt 让 pointer 可预测, 省一次 storage write。

8.4 用什么、不用什么

需求	推荐
ERC20 / ERC721 / ERC1155 / ERC4626	OZ
AccessControl / Ownable / Pausable	OZ
ReentrancyGuard	OZ (v5.2+ 用 Transient 版)
ERC20 转账	Solady SafeTransferLib (生产)
数学运算 (mulDiv、sqrt)	Solady FixedPointMathLib
大 blob 存储	Solady SSTORE2
位图 (白名单、bitmap claim)	OZ BitMaps 或 Solady LibBitmap
Merkle proof 验证	OZ MerkleProof (汇编已优化)
EIP-712 / Permit	OZ EIP712 + ERC20Permit
Proxy (UUPS / Transparent / Beacon)	OZ
ECDSA / signature recovery	OZ ECDSA 或 Solady 同名

CHAPTER 10

第 9 章 设计模式

每种模式背后都是一桩血案，或者一笔被榨干的 gas。

- The DAO (2016, \$60M) → CEI、Pull-over-Push
- Parity Multisig 双连击 (2017, 先 \$30M、后 \$280M 永久冻结) → initializer 检查、UUPS 改良
- 各种 deflationary token 让 transferFrom 失效 → SafeERC20
- USDT 长期不返回 bool → SafeTransferLib 汇编版
- Audius governance (2022, \$6M) → ERC-7201 namespace storage
- 老 token 不支持 permit → Permit2 单例
- EOA 想批量执行 → EIP-7702
- 合约超 24KB → Diamond / 模块拆分

下面 12 种模式按“重入 → 资金流 → 部署 → 升级 → 授权”的因果链组织——不是字母序的菜单，是实际工程演化路径。

9.1 CEI (Checks-Effects-Interactions)

2016 年 6 月 17 日凌晨，TheDAO 的余额每 60 秒被掏空 ~\$2M。攻击者用了一个简单到令人脸红的招式：在收到 ETH 的 fallback 里再调一次 withdraw。合约更新余额的那行代码永远到不了——因为外部调用先于状态写入。

那笔事故让以太坊硬分叉，让 ETC/ETH 永久分家。它也让 CEI 三个字母刻进每个 Solidity 工程师的肌肉记忆：Checks → Effects → Interactions。先检查参数、再改状态、最后做外部调用。违反它是 90% 重入漏洞的源头。

错例：

```
function withdraw() external {
    uint256 bal = balances[msg.sender];
    require(bal > 0);
    (bool ok,) = msg.sender.call{value: bal}(""); // ← 外部调用先于状态更新
    require(ok);
    balances[msg.sender] = 0;
}
```

攻击者 receive() 循环调 withdraw 抽干 bank。正例(CEI)：

```
function withdraw() external {
    uint256 bal = balances[msg.sender];
    require(bal > 0);
    balances[msg.sender] = 0; // ← Effects 先做
    (bool ok,) = msg.sender.call{value: bal}("");
    require(ok);
}
```


9.2 Pull-over-Push

第一年做 airdrop 的人都犯过同一个错：写一个循环 `for (i in users) users[i].transfer(amount)`，结果某个用户的智能合约钱包 fallback 默认 revert，整个 airdrop 卡死，所有人骂你。

经验总结成一句口诀：不主动给用户打钱，让他们来取。

```
mapping(address => uint256) public pendingWithdrawals;

function distributeRewards(address[] calldata users, uint256[] calldata amounts) external
onlyOwner {
    for (uint256 i = 0; i < users.length; ++i) {
        pendingWithdrawals[users[i]] += amounts[i]; // 不主动转账
    }
}

function claim() external {
    uint256 amount = pendingWithdrawals[msg.sender];
    pendingWithdrawals[msg.sender] = 0;
    (bool ok,) = msg.sender.call{value: amount}("");
    require(ok);
}
```

循环 `transfer` 时某个用户合约 revert 会卡死整个分发 (DoS)。Pull 模式让坏邻居只能伤害自己。

9.3 Factory + CREATE2

Uniswap V3 一个池子的地址，你在它部署之前就能算出来。Safe 钱包同样地址在所有链上一致。LayerZero、Permit2、Across——所有“跨链同地址”的合约都是这个魔法。

魔法的名字叫 CREATE2：地址 = `keccak256(0xff || factory || salt || keccak256(initcode))[12:]`。给定 `factory`、`salt`、`initcode`，地址完全确定。换句话说——你可以先把地址写进客户端代码，然后再部署合约。这是 Web3 工程的“未来已来”时刻。

```
contract VaultFactory {
    event VaultDeployed(address indexed vault, IERC20 indexed asset);

    function deploy(IERC20 asset) external returns (Vault v) {
        bytes32 salt = keccak256(abi.encode(asset));
        v = new Vault{salt: salt}(asset);
        emit VaultDeployed(address(v), asset);
    }

    function predict(IERC20 asset) external view returns (address) {
        bytes32 salt = keccak256(abi.encode(asset));
        bytes32 codeHash = keccak256(abi.encodePacked(
            type(Vault).creationCode,
            abi.encode(asset)
        ));
        return address(uint160(uint256(keccak256(abi.encodePacked(
            bytes1(0xff), address(this), salt, codeHash
        )))));
    }
}
```

预测地址： `addr = keccak256(0xff || factory || salt || keccak256(initcode))[12:]`，部署前可链下预知地址。

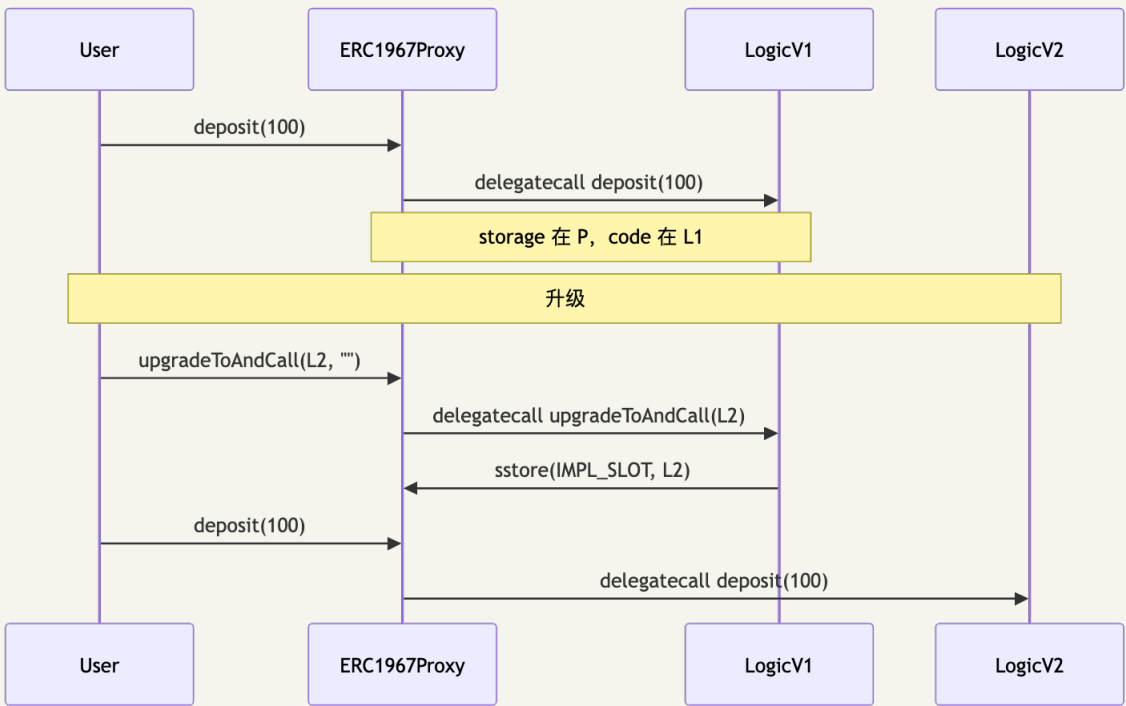
9.4 Proxy 三件套

2017 年 7 月，Parity Multisig 出过一桩诡异事故：有人调用了一个本该是 `library` 的合约的 `initWallet`，把自己设成了所有 Parity 钱包的 owner——然后他不小心 `selfdestruct` 了这个 `library`。3000 个多签钱包，5.4 亿美金，永久冻结。

Proxy 模式存在的意义就是这种"代码可改、状态不丢"的需求。但代理本身就是一个全是脚的房间——slot collision、初始化竞态、selectorrr clash、storage layout 演化、admin 权限隔离，每一处都能炸。本节把三个主流方案放在同一张桌上比较。

模式	upgrade 逻辑	gas 开销	推荐度
Transparent	代理合约里	每次 call 多 1 SLOAD	历史项目维护
UUPS	实现合约里	几乎零额外开销	新项目首选
Beacon	单独 Beacon 合约	每次多 1 SLOAD + 1 STATICCALL	一对多场景 (NFT collection)

9.4.1 UUPS 流程图



9.4.2 Storage 安全推导

UUPS 的 storage layout:

```
ERC1967._IMPLEMENTATION_SLOT = 0x360894... (hash 槽, 非 slot 0/1 这种序号槽)
ERC1967._ADMIN_SLOT          = 0xb53127... (hash 槽)
你的实现合约状态变量仍从 slot 0 开始正常铺, 与上述两个 hash 槽几乎不可能碰撞
```

ERC-1967 用特殊 slot(`keccak256("eip1967.proxy.implementation") - 1`) 存 implementation 地址, 在 keccak 域里几乎不与 storage 冲突, 实现合约可从 slot 0 正常用。

升级风险清单:

1. ❌ 新版本删除老 storage 字段: 会让 slot offset 全乱
2. ❌ 新版本调换字段顺序: 同上
3. ❌ 新版本把 uint256 改成 uint128: 会破坏 packing
4. ✅ 新版本在末尾追加字段: 吃掉 `__gap` 或 ERC-7201 namespace 内追加
5. ✅ 新版本添加 immutable: immutable 不在 storage, 安全
6. ⚠️ 新版本继承新基类: 基类 storage 会插入到子类前面, 破坏 layout——必须用 ERC-7201 隔离

OZ openzeppelin-foundry-upgrades 提供 `Upgrades.deployUUPSProxy()` + `Upgrades.upgradeProxy()`, 自动比对 storage layout 并拒绝不安全升级。

9.4.3 Selector clash 推导

Transparent 的 selector clash 解法: `msg.sender` 路由(admin 走 admin 函数, 其他全部 `delegatecall`), 代价每次多 1 SLOAD。UUPS 把 upgrade 函数放进 implementation, proxy 只做 `delegatecall`, 不存在 selector clash。

9.5 Multicall

让用户在一个 tx 里打包多次调用——前端每次调链都心疼用户多签一次的痛点解药:

```
abstract contract Multicall {
    function multicall(bytes[] calldata data) external returns (bytes[] memory results) {
        results = new bytes[](data.length);
        for (uint256 i; i < data.length; ++i) {
            (bool ok, bytes memory ret) = address(this).delegatecall(data[i]);
            if (!ok) {
                // 把内部 revert 原因冒泡上去
                assembly { revert(add(ret, 0x20), mload(ret)) }
            }
            results[i] = ret;
        }
    }
}
```

陷阱(这个坑爆过几个项目): `delegatecall` 里 `msg.sender` 仍是原始调用者(这是好事), 但 `msg.value` 每个子调用都看到完整值。如果你的 payable 子函数里写 `totalReceived += msg.value`, 攻击者发送 1 ETH 调用 5 次, 合约会以为收到 5 ETH——然后吐回 5 ETH 给攻击者, 亏 4 ETH。OZ v5 Multicall.sol 已修复, 自己写 multicall 一定要意识到这点。

9.6 Permit(EIP-2612)

每个写过 dapp 的人都被这件事烦过: 用户买一次代币要先 approve (一笔 tx, 等 ~12 秒), 再 swap (又一笔 tx, 再等 ~12 秒)。Uniswap V2 时代这是常态, 新人因此流失, 留存数据难看。

Permit 是答案。用 EIP-712 签名替代 onchain approve——签名免费、即时、不上链。一笔 tx 同时完成"授权 + 转账":

```
// 用户离线签名 (前端用 viem signTypedData)
{ owner, spender, value, nonce, deadline }
  ↓ 签名得 (v, r, s)

// 合约链上调用 (任何人都可以 relay)
token.permit(owner, spender, value, deadline, v, r, s);
token.transferFrom(owner, spender, value);
```

OZ v5 ERC20Permit 已实现, 本模块 MyToken.sol 直接继承。

陷阱(已被打过): permit 极易被 front-run。攻击者监听 mempool, 看到你的 permit 签名后抢先用同一签名调 permit ——nonce 用掉, 你接下来的 transferFrom 直接 revert。看起来"只是 DoS", 但场景一换就是大问题: 你的合约依赖 permit 成功才执行核心逻辑, front-run 一次让流程卡死。

修复: 永远把 permit + transferFrom 打包在同一笔 tx 里(用 Multicall 或路由合约一次执行), 不单独广播 permit。这是 OZ Wizard 默认生成的代码也强调的事。

9.7 Permit2(Uniswap 的通用授权层)

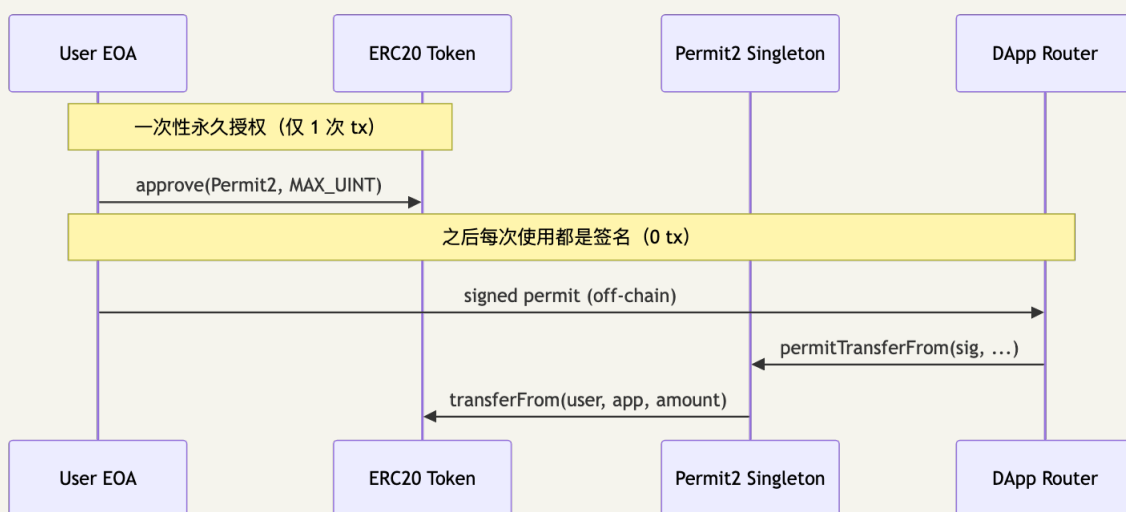
EIP-2612 有一个尴尬的硬伤: 它需要 token 自己实现 permit。USDT、WBTC、DAI 之前的几乎所有老 token 都不支持。这意味着 EIP-2612 永远只覆盖一部分用户。

Uniswap 团队 2022 年想了一个更激进的办法: 把 permit 抽出来, 做成一个所有 token 通用的中间层。Permit2 在每条主流链上通过 CREATE2 部署在同一个确定性地址

0x0000000000022D473030F116dDEE9F6B43aC78BA3 ——你可以把它当成 EVM 世界的"USB-C"。

代价是用户多一次性 onchain 操作: 先 token.approve(Permit2, MAX_UINT), 然后所有未来交互都通过 Permit2 签名。但这一次性授权撬动了所有协议共享的离线签名生态。Uniswap 自己的 universal-router、CowSwap、1inch、Across 都已迁过去。

「同一地址」是部署确定性, 不是跨链。每条链上的 Permit2 是独立合约, allowance / nonce 状态各链互不可见。同时 Permit2 的取舍是用户必须先把 token approve(Permit2, MAX_UINT) (一次性 onchain tx), 后续才能用签名授权——这是 vs EIP-2612「token 内置 permit、零 approve」的关键区别。



两个核心子合约：

- **SignatureTransfer**: 签一次用一次，最强安全。每个签名带 `nonce + deadline`，用过即销毁。
- **AllowanceTransfer**: 签名后给 app 一段时间的额度(`time-bound`)，适合频繁交互场景。

为什么越来越多走 **Permit2**: 每 token 只 approve 一次(单例共用); 所有授权有 `deadline`(根除「dormant infinite approval」); 不需 token 实现 `permit`，适用任意 ERC20。

合约接收 Permit2 转账：

```
import {ISignatureTransfer} from "permit2/src/interfaces/ISignatureTransfer.sol";

ISignatureTransfer constant PERMIT2 =
    ISignatureTransfer(0x00000000000022D473030F116dDEE9F6B43aC78BA3);

function depositWithPermit2(
    uint256 amount,
    ISignatureTransfer.PermitTransferFrom calldata permit,
    bytes calldata signature
) external {
    PERMIT2.permitTransferFrom(
        permit,
        ISignatureTransfer.SignatureTransferDetails({to: address(this), requestedAmount:
amount}),
        msg.sender,
        signature
    );
    // 此时 token 已经从 user 转到 address(this)
    _mintShares(msg.sender, amount);
}
```

新项目(DEX 路由器、跨链桥、聚合器)应直接基于 Permit2，不要求 token 实现 EIP-2612。

9.8 EIP-7702(EOA 临时变合约)

四年前 Vitalik 提出 ERC-4337 时，账户抽象需要一整套 `bundler/paymaster/entrypoint` 基础设施。EIP-7702 是更轻量的回答——Pectra 升级(2025-Q2 主网)让你的 EOA 通过签名“附身”成某个合约：一笔 tx 内执行合约逻辑，无需 4337，无需 `bundler`。

用例每天都在涌现：批量授权+`swap`+`stake`(3 tx → 1 tx); `gas` 赞助商(付 `gas` 的人和签 tx 的人分离); `session keys`(一次签名授权一段时间的动作集合)。但安全模型的颠覆程度让所有审计师都紧张——特别是 `chainId = 0` 那个雷，本节最后会展开。

Solidity 工具链支持(截至 2026-04)：

- **Foundry**: `vm.sign + vm.attachDelegation cheatcodes` 支持 EIP-7702 授权签名
- **OpenZeppelin v5.5+**: `SignerEIP7702` (v5.5 改名, v5.4 之前叫 `SignerERC7702`)，处理 `address(this) == EOA` 的签名验证
- **viem ≥ 2.21**: JS 端 `signAuthorization` API
- **Solady**: `EIP7702Proxy`，让普通 `proxy` 可以被 EOA 委托

安全红线：

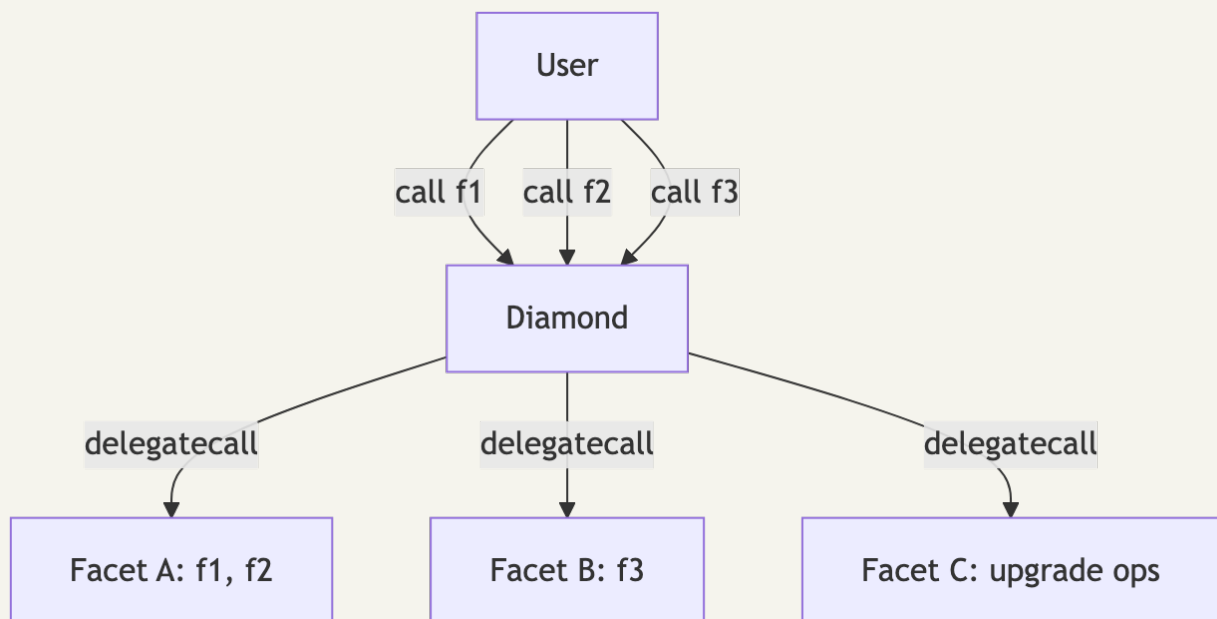
1. delegate 后所有 storage 读写发生在 EOA 地址——恶意合约可取走 EOA 持有的一切 token，比 approve 严重
2. delegation 新值直接覆盖旧值(非叠加)
3. 应用合约仍假设调用方是普通 EOA 或 4337 SmartWallet；被 EOA delegate 的合约按 ERC-1271 实现 isValidSignature
4. chainId = 0 是工具链最高危的字段：authorization tuple 允许 chainId = 0 表示「在任意链上生效」。攻击者一旦诱骗用户签 chainId=0 的 7702 auth，可在所有 EVM 链上重放、把 EOA 在每条链上都 delegate 到 sweeper。钱包/dApp 必须默认使用具体 chainId、显示拒绝 chainId=0。详见 05 模块 §11.4 攻击案例

9.9 EIP-2535 Diamond

Aavegotchi 把合约写大到了超过 24KB——以太坊单合约部署上限。怎么办？拆成多个“facet”，通过一个中央 dispatcher 按 selector 路由请求。这就是 Diamond 模式(EIP-2535)的诞生场景。

它确实优雅：突破 24KB、selector 粒度的精细升级、多团队并行开发。但代价不轻——

forge inspect 算不出 Diamond 的完整 storage layout，DiamondCut 缺 access control 时全合约可被劫持，每次调用多 1 SLOAD(查路由表)。新项目应该先问“我真的需要 Diamond 吗”，章节末有决策树。



Diamond Storage: 每个 facet 用独立 keccak256 hash 作起点，互不冲突：

```

library FacetASTorage {
    bytes32 constant SLOT = keccak256("diamond.facet.a.storage");
    struct Layout { uint256 totalSupply; mapping(address => uint256) balances; }
    function layout() internal pure returns (Layout storage l) {
        bytes32 slot = SLOT;
        assembly { l.slot := slot }
    }
}

```

优势：突破 24KB 限制；selector 粒度升级/增删；多团队并行迭代。

劣势：forge inspect 无法算 Diamond storage layout；DiamondCut 缺 access control 时全合约可被劫持；每次调用多 1 SLOAD(查路由表)；storage hash 碰撞全炸。

案例：Aavegotchi(游戏合约超 24KB)、TrueUSD(早期用，后改 UUPS)。OZ v5 推 ERC-7201 + UUPS，效果接近 Diamond 但工具链友好。

新项目决策树：

```

合约大小 < 24KB?
├─ 是 → 用 UUPS (首选)
└─ 否 → 拆模块 (factory 模式 / 多合约 + 路由合约)
      └─ 拆不了 → 才考虑 Diamond

```

EIP-2535 标准接口(强制实现)：

```

interface IDiamondCut {
    enum FacetCutAction { Add, Replace, Remove }
    struct FacetCut {
        address facetAddress;
        FacetCutAction action;
        bytes4[] functionSelectors;
    }
    function diamondCut(FacetCut[] calldata cuts, address init, bytes calldata data)
    external;
}

interface IDiamondLoupe {
    function facets() external view returns (Facet[] memory);
    function facetFunctionSelectors(address facet) external view returns (bytes4[] memory);
    function facetAddresses() external view returns (address[] memory);
    function facetAddress(bytes4 selector) external view returns (address);
}

```

`diamond-1-hardhat` / `diamond-foundry` 是参考实现仓库，不要从零写——误删 selector 即锁死合约。

9.10 Pull Payment(OZ)

ETH 转账委托给 Escrow 合约：


```
// 注意: OZ v5 已移除 contracts/security/PullPayment.sol, 需固定到 v4 引入:
// forge install OpenZeppelin/openzeppelin-contracts@v4.9.6
// 或自行从 v4 拷贝 PullPayment.sol + Escrow.sol 到本地维护。
import {PullPayment} from "@openzeppelin/contracts/security/PullPayment.sol";

contract Auction is PullPayment {
    function bid() external payable {
        if (msg.value ≤ currentBid) revert();
        // 退款给前一个出价者 (Pull 模式)
        _asyncTransfer(currentBidder, currentBid);
        currentBidder = msg.sender;
        currentBid = msg.value;
    }
}
```

_asyncTransfer 把钱放进 escrow, 受益人调 withdrawPayments 自取, 彻底隔绝 push 转账的 reentrancy 风险。

9.11 EIP-1167 Minimal Proxy(Clones)

Argent、Safe、Zerion 这些智能钱包平台每天部署上千个用户钱包, 每个钱包都是一个新合约。完整 ERC20-ish 合约部署要 500k-1.5M gas, 乘以一千个用户就是天文数字。

EIP-1167 minimal proxy(业内俗称 "clone")就是为这个场景而生: 45 字节固定 bytecode, 硬编码 implementation 地址做 delegatecall。部署成本 ~32k gas——比完整合约便宜一个数量级。代价是没有升级能力(implementation 改了影响不到已部署 clone), 但很多场景本来就不需要每个 clone 都能升级。

```
0x363d3d373d3d3d3d3d73<implementation>5af43d82803e903d91602b57fd5bf3
```

实际成本对照:

部署方式	gas
完整合约(如 ERC20)	500k - 1.5M
ERC1967Proxy(UUPS 用)	~150k
EIP-1167 minimal proxy	~32k

9.11.1 用 OZ Clones 库

```
import {Clones} from "@openzeppelin/contracts/proxy/Clones.sol";

contract VaultFactory {
    address public immutable IMPLEMENTATION;

    constructor(address impl) { IMPLEMENTATION = impl; }

    /// @notice 普通 CREATE 部署 clone
    function createVault(IERC20 asset) external returns (address vault) {
        vault = Clones.clone(IMPLEMENTATION);
        Vault(vault).initialize(asset);
    }

    /// @notice CREATE2 部署 clone (地址可预测)
    function createDeterministic(IERC20 asset, bytes32 salt) external returns (address vault) {
        vault = Clones.cloneDeterministic(IMPLEMENTATION, salt);
        Vault(vault).initialize(asset);
    }

    /// @notice 不部署只算地址
    function predictAddress(bytes32 salt) external view returns (address) {
        return Clones.predictDeterministicAddress(IMPLEMENTATION, salt, address(this));
    }
}
```

9.11.2 关键限制

1. 无 constructor: 部署后必须显式调 initialize
2. immutable 字段烧进 implementation, clone 上也是同一值
3. 不能升级: implementation 改了不影响已部署 clone
4. EXTCODESIZE 是 45: 部分反诈检查会拒绝极短代码的地址

案例: Argent/Safe wallet factory(每用户一个 clone)、Yearn v2 vault factory、Sound.xyz NFT editions、Farcaster Storage Registry。

何时选 EIP-1167 而不是 UUPS:

要求大批量部署 + 单实例不需要升级 + 共享同一逻辑 → EIP-1167
 要求大批量部署 + 全体一起升级 (一改全改) → Beacon proxy
 要求每个实例独立可升级 → UUPS (成本高, 但灵活)

9.12 EIP-3448 MetaProxy(带不可变参数的 clone)

EIP-1167 限制: 状态全靠 initialize。给 1000 个 clone 塞不同常量写到 storage 太贵。EIP-3448 把不可变参数 append 到 clone bytecode 末尾, implementation 用 helper 读取。

[标准 1167 bytecode 转发逻辑] [implementation 地址] [不可变 metadata]

9.12.1 用 Solady LibClone(OZ 未实现 EIP-3448)

```
import {LibClone} from "solady/utils/LibClone.sol";

contract PoolFactory {
    address public immutable IMPLEMENTATION;

    function createPool(address tokenA, address tokenB, uint24 fee) external returns
(address pool) {
        // 把 tokenA、tokenB、fee 编码进 clone 的 immutable args
        bytes memory args = abi.encode(tokenA, tokenB, fee);
        pool = LibClone.cloneDeterministic(IMPLEMENTATION, args, keccak256(args));
        // 不需要再调 initialize 设这三个字段
    }
}

contract Pool {
    /// @dev metadata 在 calldata 末尾, 自己读
    function tokenA() public view returns (address) {
        return abi.decode(_getImmutableArgs(0, 32), (address));
    }

    function _getImmutableArgs(uint256 offset, uint256 length) internal pure returns (bytes
memory data) {
        // Solady 的 LibClone 提供了 _getArgUint256 / _getArgAddress 等 helper
        // 这里展示原理, 实际生产请用 LibClone 的封装
    }
}
```

9.12.2 何时选 MetaProxy

大批量 clone + 每个有不同不可变常量(DEX pair/fee tier/lending 双 token)+ 不想写 storage(省 ~2100 gas/次读)。案例: Sound.xyz NFT editions、Astaria 每池利率。不用: 参数会变; 仅 1-2 个 clone(直接 immutable 更简单)。

CHAPTER 11

第 10 章 Foundry 测试体系

让我把单元/fuzz/invariant 三种测试用一句话说清楚：

单元测试只能证明你想到的输入；fuzz 能找意外输入；不变量能找意外序列。

The DAO 的重入漏洞，所有单元测试都通过了——开发者根本没想到"在 fallback 里再调一次 withdraw"这种序列。CompoundV3 的某个 edge case，是 fuzz 用 `amount = 1` 碰出来的——人写测试不会想到 1。Aave 某个清算 bug，是 invariant fuzzer 用 200 步随机调用序列碰出来的——任何人手写都不会写这种序列。

本章按"防什么 bug"的层次组织：单元 → fuzz → invariant → fork → symbolic(Halmos)。最后才是工具速查表。

10.1 四种测试形态

```
forge test --match-test test_initialState      # 单元
forge test --match-test testFuzz_             # 模糊
forge test --match-path "test/invariant/*"     # 不变量
forge test --match-path "test/fork/*" --fork-url $MAINNET_RPC_URL --fork-block-number
21000000 # fork
```

类型	输入	验证什么	何时用
单元	固定	固定输入 → 固定输出	边界条件、bug 回归
模糊(fuzz)	随机	单次调用不违反性质	算术、边界、overflow
不变量	随机调用序列	任何序列下性质成立	全局不变量(守恒、单调性)
fork	主网真实状态	与现有协议集成正确	集成测试、协议升级前演练

10.2 Foundry cheatcodes 全表(高频)

调用	作用
<code>vm.prank(addr)</code>	下一笔调用伪装 <code>msg.sender</code>
<code>vm.startPrank(addr)</code> / <code>vm.stopPrank()</code>	持续伪装
<code>vm.prank(sender, origin)</code>	同时改 <code>msg.sender</code> 和 <code>tx.origin</code>
<code>vm.deal(addr, balance)</code>	设置 ETH 余额
<code>deal(token, addr, amount)</code>	设置 ERC20 余额(forge-std 顶层)
<code>deal(token, addr, amount, true)</code>	同上但更新 <code>totalSupply</code>
<code>vm.warp(timestamp)</code>	改 <code>block.timestamp</code>
<code>vm.roll(blockNumber)</code>	改 <code>block.number</code>
<code>vm.fee(wei)</code>	改 <code>block.basefee</code>
<code>vm.chainId(id)</code>	改 <code>chainid</code>
<code>vm.expectRevert()</code>	期待下次调用 <code>revert</code>
<code>vm.expectRevert(selector)</code>	期待特定 <code>error selector</code>
<code>vm.expectRevert(abi.encodeWithSelector(...))</code>	期待带参数的 <code>error</code>
<code>vm.expectEmit(t1, t2, t3, data)</code>	期待 <code>event</code>
<code>vm.recordLogs()</code> + <code>vm.getRecordedLogs()</code>	抓所有日志
<code>vm.expectCall(addr, calldata)</code>	期待发出某次调用
<code>vm.mockCall(addr, calldata, returnData)</code>	注入返回值
<code>vm.createFork(rpc, blk)</code>	创建 <code>fork</code>
<code>vm.selectFork(id)</code>	切换 <code>fork</code>
<code>vm.snapshotState()</code>	状态快照
<code>vm.revertToState(id)</code>	回滚到快照
<code>vm.envUint("KEY")</code>	读环境变量
<code>vm.load(addr, slot)</code> / <code>vm.store(addr, slot, val)</code>	直接读写 <code>storage</code>
<code>bound(x, lo, hi)</code>	把 <code>fuzz</code> 输入限制到区间
<code>vm.assume(condition)</code>	跳过不满足条件的输入
<code>vm.skip(true)</code>	跳过整个测试
<code>makeAddr("name")</code>	由名字生成可读地址

调用	作用
<code>makeAddrAndKey("name")</code>	同时生成地址和私钥
<code>vm.sign(privKey, digest)</code>	离线签名

10.3 Handler 模式(不变量测试核心)

不变量测试的第一次实战通常长这样: 你写好不变量, 运行 `forge test`, 结果 256 轮中 240 轮都是 `revert` ——fuzzer 用各种乱七八糟的输入触发了你合约的所有边界 `require`, 没几次真正进入业务逻辑。看起来“测试通过”, 实际等于啥也没测。

Handler 模式是答案: 写一个 `wrapper` 合约, 把不可能成立的输入 `short-circuit` 掉, 让 fuzzer 的每一步都进得到核心代码。

```
contract CounterHandler is Test {
    Counter public counter;
    constructor(Counter _c) { counter = _c; }

    function inc() external { counter.inc(); }

    function dec() external {
        if (counter.count() == 0) return; // 跳过必 revert
        counter.dec();
    }
}

contract CounterInvariantTest is StdInvariant, Test {
    Counter counter;
    CounterHandler handler;

    function setUp() public {
        counter = new Counter();
        handler = new CounterHandler(counter);
        targetContract(address(handler));
    }

    function invariant_consistency() public view {
        assertEq(counter.count(), counter.increments() - counter.decrements());
    }
}
```

10.4 Ghost variables

Handler 维护独立于合约的影子状态, fuzz 结束后比对。完整示例见 10.8 节(actor pool + ghost + `fail_on_revert` + `bound`)。

10.5 fork test: 主网状态上验证集成

```
contract MainnetForkTest is Test {
    IERC20 constant USDC = IERC20(0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48);

    function setUp() public {
        vm.createSelectFork(vm.envString("MAINNET_RPC_URL"), 21_000_000);
    }

    function test_realUSDC() public {
        deal(address(USDC), alice, 1_000e6);
        assertEq(USDC.balanceOf(alice), 1_000e6);
    }
}
```

固定 block number 保证测试可复现(每天跑 mainnet HEAD 结果不稳定)。

10.6 forge coverage

```
forge coverage --report summary
forge coverage --report lcov          # 喂给 SonarQube / IDE 插件
forge coverage --ir-minimum          # 用于 stack-too-deep 的合约
```

`--ir-minimum` 解决 stack-too-deep, 但慢 3-10 倍。

10.7 gas-report 与 snapshot

```
forge snapshot          # 把所有测试 gas 写到 .gas-snapshot
forge snapshot --diff   # 与上次比较
forge test --gas-report # 详细每个函数的 gas
```

PR 流程里跑 `forge snapshot --check` , gas 涨了就拒绝 merge, 守护 gas 回归。

10.8 Handler 模式深度教学

10.8.1 Actor 池

```

contract VaultHandler is Test {
    Vault public vault;
    IERC20 public asset;

    address[] public actors;          // 演员池
    address internal currentActor;    // 当前 fuzz 步使用的演员

    /// @notice 修饰符：随机选一个演员当 msg.sender
    modifier useActor(uint256 actorSeed) {
        currentActor = actors[bound(actorSeed, 0, actors.length - 1)];
        vm.startPrank(currentActor);
        _;
        vm.stopPrank();
    }

    constructor(Vault _v, IERC20 _a) {
        vault = _v;
        asset = _a;
        // 创建 5 个有名字的演员
        for (uint256 i = 0; i < 5; ++i) {
            actors.push(makeAddr(string.concat("actor_", vm.toString(i))));
        }
    }
}

```


10.8.2 Ghost variables

```

contract VaultHandler is Test {
    // ... actor pool 同上 ...

    // ghost: 影子记账
    uint256 public ghost_totalDeposited;
    uint256 public ghost_totalWithdrawn;
    uint256 public ghost_callCount_deposit;
    uint256 public ghost_callCount_withdraw;
    mapping(address => uint256) public ghost_userDeposits;

    function deposit(uint256 actorSeed, uint256 amount) external useActor(actorSeed) {
        amount = bound(amount, 1, 1_000_000e18);

        deal(address(asset), currentActor, amount);
        asset.approve(address(vault), amount);
        vault.deposit(amount, currentActor);

        ghost_totalDeposited += amount;
        ghost_userDeposits[currentActor] += amount;
        ghost_callCount_deposit++;
    }

    function withdraw(uint256 actorSeed, uint256 amount) external useActor(actorSeed) {
        uint256 maxW = vault.maxWithdraw(currentActor);
        if (maxW == 0) return;
        amount = bound(amount, 1, maxW);

        vault.withdraw(amount, currentActor, currentActor);

        ghost_totalWithdrawn += amount;
        ghost_callCount_withdraw++;
    }
}

```

测试侧：

```

function setUp() public {
    asset = new MockERC20();
    vault = new Vault(asset);
    handler = new VaultHandler(vault, asset);

    // 限定 fuzzer 只调 handler 的这两个函数
    bytes4[] memory selectors = new bytes4[](2);
    selectors[0] = handler.deposit.selector;
    selectors[1] = handler.withdraw.selector;
    targetSelector(FuzzSelector({addr: address(handler), selectors: selectors}));

    targetContract(address(handler));
}

function invariant_solveny() public view {
    // 金库 totalAssets 必须等于 ghost 净值
    assertEq(
        vault.totalAssets(),
        handler.ghost_totalDeposited() - handler.ghost_totalWithdrawn()
    );
}

function invariant_callSummary() public view {
    // 末尾打印调用统计, 看 fuzzer 覆盖率
    console2.log("deposits:", handler.ghost_callCount_deposit());
    console2.log("withdrawals:", handler.ghost_callCount_withdraw());
}

```

10.8.3 fail_on_revert 配合 bound 的最佳实践

不变量测试的 foundry.toml 配置:

```

[invariant]
runs = 256           # 每个 invariant 跑多少独立序列
depth = 50           # 每个序列调多少次 handler
fail_on_revert = true # ← 任何 handler 调用 revert 立即失败
shrink_run_limit = 5000 # 最小化反例的尝试次数

```

fail_on_revert = true 强制 handler 内部 bound 所有输入, fuzzer 100% 调用进入业务逻辑。

10.8.4 限制 fuzzer 调用面: targetSelector / excludeContract

API	作用
targetContract(addr)	把 fuzzer 限定到这些合约
targetSelector(FuzzSelector{addr, selectors[]})	限定到这些 selector
targetSender(addr)	限定 msg.sender
excludeContract(addr)	排除这些合约(即使在 setUp 创建了也不调)
excludeSender(addr)	排除某些 sender(避免特殊角色被调)
targetArtifact("Counter")	按 artifact 名匹配

实战: `targetContract(handler) + targetSelector(...)` 配合 `actor pool + bound`。

10.9 Differential testing 与 FFI

两个独立实现相同函数应输出相同——如果不同, 至少一个有 bug。最常见: Solidity 实现与 Python/Rust 参考实现 cross-check。

10.9.1 启用 FFI

`foundry.toml` :

```
[profile.default]
ffi = true # 允许 forge 调用外部进程
```

10.9.2 实战: 测试自己写的 `sqrt` 函数

写个 Python 参考实现 `script/ref_sqrt.py` :

```
import sys
from math import isqrt
n = int(sys.argv[1])
print(hex(isqrt(n))[2:]) # 输出 hex 不带 0x
```

Foundry 测试:

```
contract SqrtDiffTest is Test {
    MyMath math;

    function setUp() public { math = new MyMath(); }

    function testFuzz_sqrtDiff(uint256 x) public {
        x = bound(x, 0, type(uint128).max);

        // 1. 调 Python 参考实现
        string[] memory cmd = new string[](3);
        cmd[0] = "python3";
        cmd[1] = "script/ref_sqrt.py";
        cmd[2] = vm.toString(x);
        bytes memory result = vm.ffi(cmd);
        uint256 expected = vm.parseUint(string(result));

        // 2. 调 Solidity 实现
        uint256 actual = math.sqrt(x);

        // 3. 必须一致
        assertEq(actual, expected, "sqrt mismatch");
    }
}
```

`forge test --ffi` 跑 256 轮, 任意反例就 fail。适用: 数学库(与 Python `mpmath` 比对)、密码学(与 OpenSSL 比对)、同一规范两个语言实现 cross-check、fork test oracle。

10.9.3 FFI 安全警告

`vm.ffi` 能跑任意 shell 命令——`ffi = true` 写在 `[profile.ci]` 而非 `[profile.default]`, 不信任的第三方测试不要跑。

10.10 Halmos: fuzz 测试即 spec 的形式化验证

a16z 的 **symbolic execution** 工具, 复用 Foundry `testFuzz_XXX` 不改一行, 用 SMT 求解器覆盖全部输入空间。

10.10.1 安装与基本用法

```

pip install halmos           # 装 a16z/halmos
halmos --function testFuzz_   # 跑所有 fuzz 测试做 symbolic
halmos --contract MyTokenTest # 限定合约

```

10.10.2 与 Foundry fuzz 的关系

工具	输入空间覆盖	速度	跨调用序列
<code>forge test fuzz</code>	256 随机点	极快	否(仅 invariant)
Halmos	所有点(符号)	慢(秒~分钟)	0.3.0 起支持 stateful invariant
Certora Prover	所有点 + 复杂 spec DSL	最慢 + 商业	是

典型 workflow: 写 fuzz 测试 → `forge test` 快速反馈 → CI 末尾跑 `halmos` (10-30 分钟, 找 fuzz 漏掉的边界) → 关键不变量用 Certora(有预算时)。

10.10.3 与 SMTChecker 的分工

工具	适用
SMTChecker	单函数 <code>require/assert</code> , 免费随手开
Halmos	整套 fuzz 测试当 spec, spec 工作量小
Certora	完整状态机, 需专业 spec writer

10.11 Fuzz dictionary 与输入控制

10.11.1 vm.assume vs bound

`vm.assume` 丢弃不满足的输入(通过率 <50% 时 fuzzer 跑空); `bound` 把任意输入映射到目标区间(零浪费, 推荐)。通过率 >90% 时 `vm.assume` 更可读。

10.11.2 makeAddr / makeAddrAndKey

```

address alice = makeAddr("alice");           // 稳定地址 + 自动 vm.label
(address bob, uint256 bobKey) = makeAddrAndKey("bob"); // 同时拿私钥用于 vm.sign

```

永远用 `makeAddr`, 不用 `address(0xBEEF)` ——trace 可读、不与合约地址冲突、同名同地址可重现。

10.11.3 过滤特殊地址

forge-std 提供 `assumeNotPrecompile(address)` 和 `assumeNotForgeAddress(address)` helper 排除特殊地址。

10.11.4 配置 fuzz seed 与 dictionary

```
[fuzz]
runs = 1024
seed = "0x1234" # 固定 seed 让 CI 可复现
dictionary_weight = 40 # 字典值出现的概率（默认 40，意味着 40% 来自字典）
include_storage = true # 把合约 storage 里的值加入字典
include_push_bytes = true # 把 bytecode 里的 PUSH 常量加入字典
```

10.12 forge 工具链全表

子命令	作用	高频用法
<code>forge build</code>	编译	<code>--via-ir</code> <code>--sizes</code> (看合约大小)
<code>forge test</code>	测试	<code>--match-test</code> <code>--match-contract</code> <code>--match-path</code> <code>-vvvv</code> <code>--gas-report</code> <code>--fuzz-runs</code>
<code>forge coverage</code>	覆盖率	<code>--report summary/lcov/debug</code> <code>--ir-minimum</code> (解决 <code>stack too deep</code>)
<code>forge snapshot</code>	gas 快照	<code>--diff</code> <code>--check</code> (CI 用)
<code>forge fmt</code>	格式化	<code>--check</code> (CI 用); 按 <code>foundry.toml</code> 的 <code>[fmt]</code> 配置
<code>forge inspect</code>	检视产物	<code><Contract> storage-layout / abi / bytecode / deployedBytecode / methods / assembly</code>
<code>forge bind</code>	生成 Rust 绑定	用 <code>alloy</code> ; TypeScript 用 <code>abigen</code> / <code>wagmi cli</code>
<code>forge create</code>	直接部署单个合约	<code>--rpc-url</code> <code>--private-key</code> <code>--constructor-args</code>
<code>forge script</code>	跑部署/运维脚本	<code>--broadcast</code> <code>--verify</code> <code>--slow</code> <code>--resume</code>
<code>forge verify-contract</code>	单独提交验证	用于 <code>forge create</code> 之后或他人合约的验证
<code>forge test --debug</code>	单步 EVM 调试	<code>forge test --debug <test_name></code> 跳到具体测试
<code>forge install</code>	装依赖	<code>pin tag:</code> OpenZeppelin/openzeppelin-contracts@v5.5.0
<code>forge remove</code>	删依赖	配合 <code>--no-commit</code>
<code>forge update</code>	升级依赖	慎用, 可能破坏 <code>pin</code>
<code>forge tree</code>	显示依赖树	看 <code>lib/</code> 全图
<code>forge clean</code>	清缓存	出问题先 <code>clean</code> 再 <code>build</code>
<code>forge geiger</code>	标记不安全 cheatcode 使用	审计辅助
<code>forge doc</code>	从 NatSpec 生成文档	<code>--build</code> <code>--serve</code>
<code>forge selectors</code>	列出所有 selector	<code>find --signatures</code> 反查名字

10.12.1 forge create vs forge script vs forge deploy

命令	适用
forge create	单个合约快速部署
forge script	复杂部署：多合约、有逻辑、跨链(推荐)
forge deploy	2026 新增声明式部署(alpha)

99% 的项目用 `forge script`。

10.12.2 forge inspect 极其有用的子命令

```
forge inspect MyToken storage-layout      # JSON 列出每个变量的 slot/offset/type
forge inspect MyToken abi                  # 提取 ABI
forge inspect MyToken methods              # 函数 selector 表
forge inspect MyToken assembly             # 汇编输出 (看编译器优化结果)
forge inspect MyToken deployedBytecode    # 部署后字节码
forge inspect MyToken --pretty methods    # 表格形式
```

`storage-layout` 是升级安全检查的命脉，加进 CI: 升级前对比新旧 layout，不兼容直接拒绝 merge。

CHAPTER 12

第 11 章 部署与运维

代码写完不算交付，部署上链且能被人验证才算。我见过 testnet 上跑得风生水起的协议，mainnet 部署时 etherscan 验证失败——用户看到的是一坨 bytecode，没人敢交互。

这一章是把上面 10 章产物推到链上的工具表：可复现的部署脚本、Etherscan 自动验证、Safe 多签升级流程、anvil 本地复现 mainnet 状态。每一项都有踩坑成本，能省就省。

11.1 forge script

部署逻辑写成 Solidity 脚本：

```
contract DeployScript is Script {
    function run() external returns (MyToken token) {
        uint256 pk = vm.envUint("PRIVATE_KEY");
        vm.startBroadcast(pk);
        token = new MyToken("MyToken", "MTK", 1_000_000 ether, vm.addr(pk));
        vm.stopBroadcast();
    }
}
```

```
forge script script/DeployScript.s.sol:DeployScript \
  --rpc-url $SEPOLIA_RPC_URL \
  --private-key $PRIVATE_KEY \
  --broadcast --verify -vvvv
```

关键 flag：--broadcast（真上链）、--verify（自动 etherscan 验证）、-vvvv（最详细日志）、--slow（等上链再发下一笔，避免 nonce 冲突）、--resume（继续中断的脚本）。

11.2 私钥管理：从开发到生产

场景	推荐
本地 dev	--private-key \$PRIVATE_KEY（写 .env, git ignore）
Testnet	cast wallet import 生成加密 keystore, --account name 用
Mainnet 部署	硬件钱包(Ledger / Trezor) --ledger flag
多签部署	Safe + forge script --no-broadcast --sig "run()" 生成 calldata, 再 Safe 签名
CI 自动	KMS / Doppler / GCP Secret Manager 注入

真实私钥永远不提交 git。 .env.example 是模板， .env 在 .gitignore 。

11.3 cast: Web3 命令行瑞士军刀

```
# eth_call (只读)
cast call 0x... "balanceOf(address)(uint256)" 0xYou
cast call 0x... "balanceOf(address)(uint256)" 0xYou --block 21000000

# eth_sendTransaction (上链)
cast send 0x... "transfer(address,uint256)" 0xRcv 1ether \
  --rpc-url $SEPOLIA_RPC_URL --private-key $PK

# 解析 selector / signature
cast 4byte 0xa9059cbb          # → "transfer(address,uint256)"
cast sig "transfer(address,uint256)" # → 0xa9059cbb

# 直接读 storage slot
cast storage 0x... 0
cast storage 0x... 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc

# 抓事件
cast logs --address 0x... --from-block 18000000 \
  --rpc-url $RPC \
  "Transfer(address,address,uint256)"

# 计算 EIP-712 hash
cast eip712-hash ...

# 跟踪某笔 tx
cast run 0txHash --rpc-url $RPC

# 估 gas
cast estimate 0x... "deposit(uint256)" 1000

# 获取链上代码
cast code 0x... > deployed.bin
```

11.4 anvil: 本地链

```
anvil --host 0.0.0.0 --chain-id 31337
anvil --fork-url $MAINNET_RPC_URL --fork-block-number 21000000
anvil --gas-price 0 --base-fee 0 # 测试用, 免 gas
```

默认 10 个账户各 10000 ETH。--state state.json 持久化状态重启不丢。

CHAPTER 13

第 12 章 Gas 优化清单

我见过一个项目花两周时间用汇编重写了一个查询函数，省了 800 gas——结果用户根本不调那个函数。也见过另一个项目把所有 `require` 改成自定义错误，单笔交互省了 4000 gas，每月省下数千刀的用户开支。

经验法则：永远先 **profile** 再优化。打开 `forge test --gas-report`，看哪个函数被调最多 + 单次 gas 最高，那才是 ROI 入口。下面按 ROI 从高到低排：

Tier 1: 必做

1. `external + calldata` 替代 `public`
2. 自定义错误替代 `require(c, "string")`
3. `immutable / constant` 替代 `storage`
4. `storage` 变量打包(多个 ≤ 32 字节字段塞同一 slot)
5. 循环内缓存 `storage` 到 `memory`
6. `unchecked` 包裹安全算术(循环计数器、已检查过的减法)

Tier 2: 常做

1. 位图替代 `mapping(uint  $\Rightarrow$  bool)` (256 bool 塞 1 slot)
2. `transient storage` 锁替换 `ReentrancyGuard`
3. `event` 替代 `storage`(链下索引、不需链上读的历史数据)

Tier 3: 场景性

1. `SSTORE2/SSTORE3`: 大块 `immutable` 数据
2. `Solady` 替换 `OZ: SafeTransferLib`、`FixedPointMathLib`、`LibBitmap`
3. 手写 `Yul`: 仅热路径，必须额外测试

Gas 实测(本模块附带)

```
cd code/
forge snapshot
cat .gas-snapshot
```

预期对照：

```
testFuzz_mint_underCap (runs: 256,  $\mu$ : 75_932,  $\sim$ : 76_521)
test_permit (gas: 122_491)
testFuzz_transfer_conserves (runs: 256,  $\mu$ : 105_872,  $\sim$ : 106_244)
```

把 `MyToken.sol` 的 `_transfer` 替换成 `Solady SafeTransferLib`，预期 gas 从 $\sim 30k$ 降到 $\sim 22k$ (省 27%)。

反优化警告

- 为省 1-2 SLOAD 把代码写成无法读——审计成本会回头吃掉你
 - `via_ir = true` 大合约省 5-15%，但编译慢 5-10 倍，且 0.8.28~0.8.33 有清空 bug
 - 先 `forge test --gas-report` 找瓶颈，再针对性优化
-

CHAPTER 14

第 13 章 框架选型与备选语言生态

2018 年大家用 Truffle, 2020 年迁到 Hardhat 2, 2022 年开始 Foundry, 2025 年又出了 Hardhat 3 ——Web3 工具链每两年地壳运动一次。本章不告诉你"必选哪个", 而是告诉你各家什么情况下选什么, 以及哪些语言/框架现在已经退场(Brownie)、哪些是只在特殊场景才用(Huff、Stylus)。

13.1 Foundry vs Hardhat 3(2026 现状)

Hardhat 3(2025-12 production-ready beta)重写了 Rust 核心, 支持原生 Solidity 测试、原生 fuzz、多链/OP Stack 模拟。

维度	Foundry(v1.x)	Hardhat 3(beta, prod-ready)	Hardhat 2 (legacy)
测试语言	Solidity	Solidity(新) + TypeScript	TypeScript / JS
测试速度	极快(Rust)	接近 Foundry(Rust 重写)	慢
Fuzz	原生	原生(新)	插件, 弱
Invariant	原生 + handler 模式	部分支持, 仍演进	几乎没有
Cheatcodes	极丰富	兼容大部分 forge cheatcodes	较弱
部署脚本	Solidity (forge script)	TypeScript(默认)+ Solidity(可选)	TypeScript
多链/OP Stack	单链 fork	原生多链 + OP Stack 模拟	单链
配置	TOML	ESM TypeScript(新)	CJS JS
EIP-7702 cheatcodes	已支持	已支持	不支持
gas snapshot	forge snapshot	仍在开发	插件
生态插件	forge-std + Solady	重写中(部分插件还没移植)	巨多
学习曲线	中等	中等(变化大)	低
团队语言	Solidity 强	Solidity + TS 双栈	TS

推荐: 新项目用 Foundry(Uniswap V4/Aave V4/Lido/Morpho/EigenLayer 全 Foundry); TS 重度团队用 Hardhat 3(前端/OP Stack 模拟); HH2 → HH3 是重写级迁移, 有条件建议直接迁 Foundry。混合: Foundry 做合约测试, Hardhat 3 做前端集成(hardhat-foundry 共享 remappings)。

13.2 Foundry 工具链子项目对照

工具	角色	类比
forge	编译 + 测试 + 部署脚本	npm + jest + hardhat-deploy
anvil	本地链	ganache / hardhat node
cast	RPC 命令行	web3.js cli + ethers cli
chisel	Solidity REPL	python repl

Foundry v1.0(2025-02 发布)转入 stable, 2026 仍 nightly 滚动。foundryup --version stable 装稳定版, foundryup 默认 nightly。

13.2.1 Hardhat 3 production beta 实测对比

同一 ERC20 + ERC4626 + UUPS 项目实测：

操作	Foundry v1.x	Hardhat 3 beta	Hardhat 2
冷启动 + build	~2.1s	~3.8s	~9.5s
forge test / hardhat test (Solidity 测试)	~1.4s(256 fuzz runs)	~1.9s	N/A(无原生 Sol 测试)
Hardhat test (TS 测试)	N/A	~2.7s	~14s
Coverage(同覆盖率)	~5s	~8s	~45s
Fuzz 256 runs	0.8s	1.2s	不支持
写部署脚本	Solidity(forge script)	Solidity 或 TS	TS
多链 fork(OP+Mainnet+Arb 同时)	手工配 3 个 fork	原生 multichain config	手工

HH3 亮点：Solidity + TS 双栈测试、OP Stack 原生模拟(chainType: "optimism")、ESM-first、声明式插件。

HH3 截至 2026-04 短板：无原生 gas snapshot、ledger plugin 未移植、invariant handler 模式不如 Foundry 成熟。

13.3 备选语言(按生产成熟度)

99% 的新项目仍用 Solidity, 替代品用于读主网合约、极致性能或跨 VM 场景。

Vyper(Curve / Lido 在用)

Python-like EVM 语言, v0.4.3(2026-Q1), 默认 EVM prague 。优势：不支持继承/modifier/内联汇编(审计员的福音)；0.4.2 起取消 @nonreentrant 装饰器的内置语义、改成显式调用 nonreentrant modifier(语义更直观、避免编译器实现 bug 重演)；Venom 优化器在数学合约 gas 比 Solidity 低。劣

势：工具链弱；2023-07 Curve hack 由 0.2.15-0.3.0 reentrancy lock 编译器 bug 导致 ~\$70M 损失；无 OZ 等价库。何时选：数学密集+逻辑简单(AMM)+Python 团队。

Huff(极致 gas 优化)

Aztec 团队的 macro 汇编语言，直接暴露 EVM stack，无变量无类型。

```
#define macro MAIN() = takes(0) returns(0) {
    0x00 calldataload 0xE0 shr    // load selector
    dup1 __FUNC_SIG(transfer) eq transfer jumpi
    dup1 __FUNC_SIG(approve) eq approve jumpi
    0x00 0x00 revert
    transfer:
        // ... 用栈直接操作 ...
}
```

省 30-80% gas，唯一能接近手工 EVM 字节码的语言。不可读/不可审计/无溢出检查。仅用于 MEV 套利、核心 router 热路径，绝不写业务逻辑。

Yul

Solidity IR 兼 assembly { } 块，100% 互操作，非独立语言(详见第 7 章)。

Fe(实验中)

EF 孵化的 Rust-like 语言，alpha，不生产可用。

Solang(Polkadot / Solana / Stellar)

Solidity → 非 EVM 的 LLVM 编译器，v0.3.4(2026-04)，兼容 0.8.x 大部分语法，但链差异需改写。

Sway(Fuel 网络)

Fuel Labs 的 Rust-like 语言，FuelVM 专用，2026-04 仍 testnet(beta-5)。仅确定做 Fuel 项目时关注。

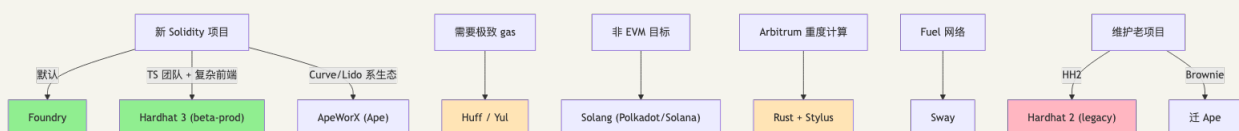
Rust + Stylus(Arbitrum WASM 合约)

生产可用，Stylus SDK v0.10(2026-04)。Arbitrum 主网支持 WASM + EVM 共存，Solidity 可调用 Rust 合约。优势：Rust 全生态可用；复杂数学 gas 比 Solidity 快 10-100 倍；类型系统+所有权安全。劣势：仅 Arbitrum One/Nova；OZ rust-contracts-stylus 仍演进中。选用条件：项目部署 Arbitrum + 有复杂数学/密码学 + 团队有 Rust 经验。

13.4 备选 Python 框架(Brownie / Ape)

Brownie 2024 停维，社区迁移到 ApeWorX (Ape)(主要用户：Curve/Yearn/yAcademy)。Vyper 一等公民，Python 数据科学栈集成，但 fuzz/invariant 弱于 Foundry。新项目仍推荐 Foundry。

13.5 一张图看清 2026 框架格局



绿色 = 新项目首选；橙色 = 特定场景；粉色 = 仅维护，不新建。

13.6 调试器与 IDE 集成

工具	用途
<code>forge test --debug <test></code>	step-by-step EVM 调试器, 跳转到任意 opcode
<code>cast run <txHash></code>	重放主网 tx, 显示完整 trace
Hardhat 3 <code>console.log</code>	TypeScript 端 print 调试
Foundry <code>console2.log</code>	Solidity 端 print(forge-std 提供)
Tenderly	视觉化 debugger, 主网 tx 友好分析
Slither	静态分析(access control、reentrancy、arithmetic)
Mythril / Manticore	符号执行(学术界用得更多, 工业用 fuzz 替代)
Echidna	属性 fuzz(与 Foundry invariant 类似但更老牌)

VSCode: Solidity by Juan Blanco + Hardhat for Visual Studio Code(Nomic Foundation, Foundry 项目也用)。Cursor 默认带这两个。

CHAPTER 15

第 14 章 AI 辅助开发：实际生产力增益

2024 年出过一桩名为 "AI rugged me" 的事故：开发者复制粘贴 ChatGPT 给的 ERC20 实现，缺 `onlyOwner`，部署 5 分钟后被人 mint 走全部 supply。这不是个例——AI 生成的合约里最常见的 bug 是访问控制缺失，第二常见是 `transferFrom` 不包 `SafeERC20`，第三是 `tx.origin` 当鉴权。

但禁用 AI 也不是答案。模型在样板代码、测试铺量、注释翻译这些事上能让你快 3 倍。问题是要知道它在哪些事上靠谱、哪些事上一定要审视。

14.1 AI 增益 (2026)

模型不知道你升级了 OZ v5，也不会跑 `forge test --gas-report`。加速：样板代码 (OZ 继承)、测试铺量、接口草稿、注释改写、fork test 脚手架、批量重构。不可信：安全设计决策、协议深度集成 (常用过时 ABI)、gas 优化 (不跑 benchmark)、不变量设计。

14.2 AI 生成代码最常见的 5 类错误

错误 1：缺 access control

AI 生成的 `mint` / `setFee` / `withdraw` / `pause` 经常缺 `onlyOwner` 或 `onlyRole`，部署即被白嫖。

```
// AI 生成 (错)
function mint(address to, uint256 amount) external {
    _mint(to, amount);
}

// 你应该改成
function mint(address to, uint256 amount) external onlyRole(MINTER_ROLE) {
    _mint(to, amount);
}
```

错误 2：直接用 `transfer` / `transferFrom` 不包 `SafeERC20`

USDT 等非标 token 不返回 bool，会让合约对部分主流币失效。

```
// AI 生成 (错)
IERC20(token).transferFrom(user, address(this), amount);

// 你应该改成
using SafeERC20 for IERC20;
IERC20(token).safeTransferFrom(user, address(this), amount);
```

错误 3：`tx.origin` 当鉴权

钓鱼合约可绕过。永远用 `msg.sender`。

错误 4：`block.timestamp` 当随机数

出块者可操控 ±15 秒。修复：Chainlink VRF 或 RANDAO。

错误 5: 升级合约时 storage 顺序破坏

```
// V1
contract Token { uint256 a; uint256 b; }

// V2 (AI 生成, 错)
contract TokenV2 { uint256 b; uint256 a; } // 顺序换了!

// V2 (正确)
contract TokenV2 { uint256 a; uint256 b; uint256 c; } // 只在末尾追加
```

OZ Upgrades.upgradeProxy() 自动检测并拒绝不安全升级, 永远用它代替手动 upgradeToAndCall 。

14.3 workflow 推荐

工具: Cursor(日常补全)、Claude Code/Codex CLI(大任务)、Aider(git diff 迭代)、Continue.dev+本地模型(隐私)。审视 AI 输出: ① forge build && forge test ; ② slither . ; ③ 每个 external 函数走攻击者视角。

14.4 真实工程师 workflow (一周的样子)

阶段	工具	时间占比
需求理解、画图	笔 + AI 头脑风暴	15%
写接口和数据模型	手写 + AI 提供模板	10%
实现合约	Cursor 补全 + 手工把控关键逻辑	30%
写单元测试	AI 派生 case + 手工补 edge	20%
写不变量测试	全手工 (AI 不擅长)	10%
审视 + 重构 + gas profile	手工为主	10%
部署 + 集成	forge script + cast	5%

AI 把前 4 项时间砍一半, 省下来的应投给后 3 项——那才是工程师真正创造价值的地方。

14.5 静态分析与形式化工具对比

工具	类型	语言	速度	误报率	何时用
Slither	静态分析	Python	中	中(需调)	默认必跑; CI 标配
Aderyn	静态分析	Rust	极快	低(新)	大型仓库快速扫描
Mythril	符号执行	Python	慢	高	审计辅助
Manticore	符号执行	Python	极慢	高	学术
SMTChecker	形式化	编译器内置	中	低	写代码时随手开
Halmos	形式化	Python(用 z3)	慢	极低	fuzz 测试做 spec
Certora Prover	形式化	商业 DSL	最慢	极低	高价值协议
Echidna	属性 fuzz	Haskell	中	中	Foundry invariant 替代

14.5.1 Slither(必装)

```
pip install slither-analyzer
slither . # 全部检测器
slither . --exclude-low # 排低严重度
slither . --checklist # Markdown 输出 (适合 PR comment)
```

能找到(90+ 检测器): reentrancy、未初始化 storage、suicidal/locked-ether、任意 call destination、tx.origin 鉴权、unchecked return value、错误 access control、storage layout collision。

14.5.2 Aderyn(Cyfrin Rust 重写)

```
cargo install aderyn
aderyn . # 跑所有
aderyn . -o report.md # 输出 Markdown
aderyn . -o report.json # JSON 给 IDE 集成用
```

Aderyn vs Slither(2026-04 现状):

维度	Slither	Aderyn
速度	中	快 5-10 倍
检测器数量	90+(成熟)	50+(成长中)
自定义检测器	Python 写	Rust 写(性能好)
集成	Foundry/Hardhat	Foundry/Hardhat
VSCode 插件	有	有官方插件
误报率	中	低

实战: 两个都跑。CI fail-on-finding 用 Aderyn, advisory 用 Slither。

14.5.3 工具组合的标准 CI pipeline

```
# .github/workflows/ci.yml
- run: forge fmt --check
- run: forge build --sizes # 检查合约大小限制
- run: forge test -vvv
- run: forge snapshot --check # gas 回归
- run: forge coverage --report summary
- run: aderyn . # 静态分析
- run: slither . --exclude-low --exclude-informational
- run: halmos --function testFuzz_ # symbolic (可选, 慢)
```

14.6 Uniswap V2 Pair walkthrough

读源码是最被低估的学习方式。Uniswap V2 Pair 是我推荐每个新人精读三遍的那一份: 约 200 行, 2020 年部署后 5 年没改一行代码, 处理过数千亿美金交易。它的每一行都有"为什么这么写"的故事——为什么 reserve 用 uint112、为什么有个 "MINIMUM_LIQUIDITY=1000" 的常量、为什么 swap 先转账后检查 K。

源码: [Uniswap/v2-core/UniswapV2Pair.sol](https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2Pair.sol), 是"最小完整 AMM"的教科书实现。下面我们逐节拆。

14.6.1 Storage 解读

```

contract UniswapV2Pair is UniswapV2ERC20 {
    using SafeMath for uint;
    using UQ112x112 for uint224;

    uint public constant MINIMUM_LIQUIDITY = 10**3; // 永久销毁的 LP, 防 inflation attack
    bytes4 private constant SELECTOR =
    bytes4(keccak256(bytes('transfer(address,uint256)'))));

    address public factory; // slot N+0: 哪个 factory 部署的 (address 占 20 字节, 剩余
    12 字节装不下下一个 address)
    address public token0; // slot N+1 (独占: 与 factory 共占 40 字节超过 32, 无法
    packing)
    address public token1; // slot N+2
    uint112 private reserve0; // slot N+3 offset 0
    uint112 private reserve1; // slot N+3 offset 14 (112 bit = 14 字节)
    uint32 private blockTimestampLast; // slot N+3 offset 28 (剩下的 4 字节) → 整个 slot
    满
    uint public price0CumulativeLast; // slot N+4: TWAP 累积
    uint public price1CumulativeLast; // slot N+5
    uint public kLast; // slot N+6: 上次 mint fee 时的 K = reserve0 *
    reserve1
    uint private unlocked = 1; // slot N+7: reentrancy lock
}

```

设计精华: ① reserve0/reserve1/blockTimestampLast 三字段打包 1 个 slot, swap 写 1 次 SSTORE 而非 3 次(省 ~10000 gas/swap); ② uint112 上限 $2^{112}-1 \approx 5.2e33$, 18 decimals 下够用; ③ uint32 blockTimestamp 到 2106 年才溢出。

14.6.2 K 不变量

reserve0 * reserve1 ≥ k_old (swap fee 加成后必须保持或增加):

```

function swap(uint amount0Out, uint amount1Out, address to, bytes calldata data) external
lock {
    require(amount0Out > 0 || amount1Out > 0);
    (uint112 _reserve0, uint112 _reserve1,) = getReserves();
    require(amount0Out < _reserve0 && amount1Out < _reserve1);

    uint balance0;
    uint balance1;
    {
        // optimistic transfer: 先打钱再要求用户转入
        if (amount0Out > 0) _safeTransfer(token0, to, amount0Out);
        if (amount1Out > 0) _safeTransfer(token1, to, amount1Out);
        if (data.length > 0) IUniswapV2Callee(to).uniswapV2Call(msg.sender, amount0Out,
amount1Out, data); // flash loan
        balance0 = IERC20(token0).balanceOf(address(this));
        balance1 = IERC20(token1).balanceOf(address(this));
    }
    uint amount0In = balance0 > _reserve0 - amount0Out ? balance0 - (_reserve0 -
amount0Out) : 0;
    uint amount1In = balance1 > _reserve1 - amount1Out ? balance1 - (_reserve1 -
amount1Out) : 0;
    require(amount0In > 0 || amount1In > 0);
    {
        // 0.3% fee: 把 input 乘以 997 再除以 1000
        uint balance0Adjusted = balance0.mul(1000).sub(amount0In.mul(3));
        uint balance1Adjusted = balance1.mul(1000).sub(amount1In.mul(3));
        require(
            balance0Adjusted.mul(balance1Adjusted) ≥
uint(_reserve0).mul(_reserve1).mul(1000**2),
            'UniswapV2: K'
        );
    }
    _update(balance0, balance1, _reserve0, _reserve1);
    emit Swap(msg.sender, amount0In, amount1In, amount0Out, amount1Out, to);
}

```

学习点：① optimistic transfer: 先转 output 再检查 K, callback 内可做任意事(flash loan 还款), 最终满足 K 即可；② flash loan 是 2 行 callback 的免费副产物；③ K 检查用 $\geq$ (多打入变 dust, 少打入卡 require)；④ 乘 1000 再算——把 0.3% fee 转成整数除法(EVM 无浮点)。

14.6.3 Mint: 流动性铸造与 inflation attack 防御

```
function mint(address to) external lock returns (uint liquidity) {
    (uint112 _reserve0, uint112 _reserve1,) = getReserves();
    uint balance0 = IERC20(token0).balanceOf(address(this));
    uint balance1 = IERC20(token1).balanceOf(address(this));
    uint amount0 = balance0.sub(_reserve0);
    uint amount1 = balance1.sub(_reserve1);

    bool feeOn = _mintFee(_reserve0, _reserve1);
    uint _totalSupply = totalSupply;
    if (_totalSupply == 0) {
        liquidity = Math.sqrt(amount0.mul(amount1)).sub(MINIMUM_LIQUIDITY);
        _mint(address(0), MINIMUM_LIQUIDITY); // ★ 永久销毁 1000 wei LP
    } else {
        liquidity = Math.min(amount0.mul(_totalSupply) / _reserve0,
            amount1.mul(_totalSupply) / _reserve1);
    }
    require(liquidity > 0, 'UniswapV2: INSUFFICIENT_LIQUIDITY_MINTED');
    _mint(to, liquidity);
    _update(balance0, balance1, _reserve0, _reserve1);
    if (feeOn) klast = uint(reserve0).mul(reserve1);
    emit Mint(msg.sender, amount0, amount1);
}
```

关键：① MINIMUM_LIQUIDITY=1000 wei 永久销毁给 address(0)，让攻击者无法把 share 操纵到 0；② 首次 mint LP = $\sqrt{x*y}-1000$ （几何平均，与价格比无关）；③ 之后 mint 取 $\min(\dots)$ （按比例，防单边攻击）。

14.6.4 设计模式总结

- Lock modifier (V2 用 storage, V4 改 transient 省 5000 gas/call)
- 3 字段挤 1 slot: SSTORE 优化教科书
- optimistic + post-check: 天然支持 flash loan
- $K = \text{constant product}$: 全部金融逻辑就一个不等式
- MINIMUM_LIQUIDITY: 销毁 1000 wei 防 inflation attack (同思路: ERC4626 `_decimalsOffset`)
- 先打钱再读 balance: 自然处理 fee-on-transfer token

14.6.5 推荐的下一个 walkthrough

1. Solady ERC20 (80 行): 汇编版 ERC20, 对照 OZ 看两种风格
2. Morpho Blue (650 行): 最简 lending market, 无治理、单合约、纯数学
3. Uniswap V4 PoolManager: transient storage + hooks 极致用法
4. Aave V3 Pool: reserve cache、利率曲线、清算

每个 walkthrough 问自己: storage 布局为何如此? 不变量是什么? 攻击者能做什么? 经济假设是什么? 能借鉴什么?

CHAPTER 16

第 15 章 实战项目

读到这里，理论部分已经走完。下面这套代码你可以 clone 下来，跑一遍 `make test`，亲手感受 fuzz/invariant/fork 三层测试如何配合，UUPS 升级在 Foundry 里如何端到端验证。骨架小但麻雀俱全：ERC20 + Permit + Burnable + AccessControl + UUPS——和 OZ Wizard 一键生成的同结构。

```
04-Solidity开发/code/
├── foundry.toml      # solc 0.8.28, evm cancun, fuzz/invariant 默认参数
├── remappings.txt    # OZ / forge-std / solady 别名
├── Makefile          # install / test / deploy 一键
├── .env.example      # 配置模板
├── src/
│   ├── MyToken.sol   # ERC20 + ERC20Permit + ERC20Burnable + AccessControl
│   ├── MyTokenUUPS.sol # UUPS 可升级 ERC20 (含 V2 用于升级测试)
│   └── Counter.sol    # 不变量测试用最小合约
├── test/
│   ├── MyToken.t.sol  # 单元 + 模糊测试
│   ├── MyTokenUUPS.t.sol # UUPS 升级流程端到端测试
│   ├── invariant/CounterInvariant.t.sol # Handler + invariant
│   └── fork/MainnetFork.t.sol          # USDC 主网 fork
└── script/
    └── DeployScript.s.sol # 同时部署 ERC20 与 UUPS
```

15.1 一键启动

```
cd 04-Solidity开发/code

# 1. 安装依赖 (pin 到具体 tag, 可复现)
make install

# 2. 编译
make build

# 3. 跑全部测试 (不含 fork)
make test

# 4. 跑 fork 测试 (需要 .env 里的 MAINNET_RPC_URL)
cp .env.example .env && vi .env # 填好 RPC
source .env && make test-fork

# 5. 看 storage layout
make storage

# 6. 部署到 sepolia
make deploy-sepolia

# 7. 启动本地链
make anvil
```

15.2 make test 期待输出

```
Ran 7 tests for test/MyToken.t.sol:MyTokenTest
[PASS] test_initialState() (gas: 18234)
[PASS] test_mint_byMinter() (gas: 76521)
[PASS] test_revert_mint_byNonMinter() (gas: 17033)
[PASS] test_revert_mint_overCap() (gas: 19872)
[PASS] test_permit() (gas: 122491)
[PASS] testFuzz_mint_underCap(uint256) (runs: 256,  $\mu$ : 75932,  $\sim$ : 76521)
[PASS] testFuzz_transfer_conserve(uint256,uint256) (runs: 256,  $\mu$ : 105872,  $\sim$ : 106244)

Ran 4 tests for test/MyTokenUUPS.t.sol:MyTokenUUPSTest
[PASS] test_initialize_setsOwner() (gas: 22118)
[PASS] test_revert_doubleInitialize() (gas: 18233)
[PASS] test_upgradeToV2_preservesState() (gas: 1284733)
[PASS] test_revert_upgrade_byNonOwner() (gas: 1098722)

Ran 2 invariant tests for test/invariant/CounterInvariant.t.sol:CounterInvariantTest
[PASS] invariant_countConsistency() (runs: 256, calls: 8192, reverts: 0)
[PASS] invariant_noUnderflow() (runs: 256, calls: 8192, reverts: 0)
```

15.3 关键代码导读

MyToken.sol —— OZ v5 多重继承正确顺序：

```
contract MyToken is ERC20, ERC20Burnable, ERC20Permit, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    error MintCapExceeded(uint256 requested, uint256 cap);
    uint256 public immutable MINT_CAP;

    constructor(string memory n, string memory s, uint256 cap, address admin)
        ERC20(n, s) ERC20Permit(n)
    { MINT_CAP = cap; _grantRole(DEFAULT_ADMIN_ROLE, admin); _grantRole(MINTER_ROLE, admin); }

    function mint(address to, uint256 amount) external onlyRole(MINTER_ROLE) {
        if (amount > MINT_CAP) revert MintCapExceeded(amount, MINT_CAP);
        _mint(to, amount);
    }
}
```

MyTokenUUPS.sol —— UUPS 三点：① `_disableInitializers()` 在 constructor；②

`_authorizeUpgrade` override 加 `onlyOwner`；③ `__gap` 预留升级空间。

CounterInvariant.t.sol —— Handler 模式(见 10.3 节)。**MainnetFork.t.sol** —— deal cheatcode 灌余额 + try/catch CI 友好写法。

CHAPTER 17

第 16 章 习题与参考解答

下面 6 道题不是为了考你——是把前 15 章的概念压成 6 个手感场景。从 ERC4626 inflation attack 到 UUPS 5 个不易察觉的安全坑、从重入修复到 Solady benchmark, 每题都有“为什么这么做”的答辩。骨架在 `exercises/` 目录, 建议先自己做再查参考解答——卡 30 分钟比直接看答案多记 10 倍。

习题 1(基础): ERC-4626 收益金库

ERC-4626 第一年公开后, 多位审计师独立报告了同一个攻击: **inflation attack**。攻击者在你的金库刚部署、`totalSupply=0` 的瞬间存 1 wei, 然后直接给金库地址 `transfer 1e18 underlying`。等下一个真用户存 100 USDC 时, 份额计算 $100e6 * 1 / 1e18 = 0$ 取整为 0——用户钱进去了, 份额是 0, 攻击者全吞。

防御办法是 OZ ERC4626 的 `_decimalsOffset()` ——把份额放大 $1eN$ 倍, 攻击者要把比例操纵到能“吃掉”用户份额, 捐款金额会涨到经济不可行。本题让你实操一遍。

基于 OZ v5 ERC4626 实现 `YieldVault`: ① 任意 ERC20 资产; ② `_withdraw` 收 0.5% 提现费; ③ `_decimalsOffset()` 返回 6 防 inflation attack; ④ 测试覆盖: 单元(`deposit/withdraw/mint/redeem`)+ 模糊(单调性)+ 不变量(`totalAssets >= sum(balanceOf)`)。

骨架(完整版在 `exercises/01-erc4626-vault/`):

```
contract YieldVault is ERC4626 {
    address public immutable feeRecipient;
    uint256 public constant FEE_BPS = 50; // 0.5%
    uint256 public constant BPS = 10_000;

    constructor(IERC20 asset_, address feeRecipient_)
        ERC4626(asset_) ERC20("YieldVault", "yv")
    { feeRecipient = feeRecipient_; }

    function _decimalsOffset() internal pure override returns (uint8) { return 6; }

    function _withdraw(
        address caller, address receiver, address owner,
        uint256 assets, uint256 shares
    ) internal override {
        uint256 fee = (assets * FEE_BPS) / BPS;
        super._withdraw(caller, feeRecipient, owner, fee, 0); // 把 fee 转给 recipient
        super._withdraw(caller, receiver, owner, assets - fee, shares);
    }
}
```

inflation attack 防护: 存 1 wei 再「捐」1e18 使 `totalAssets >> totalSupply`, 下一个存款者份额取整为 0。 `_decimalsOffset=6` 把比例放大 $1e6$ 倍, 攻击者需捐 $1e24$ wei——经济不可行。

习题 2(基础): 50 个变量的 storage 排布

见 `exercises/02-storage-layout/`, 重排到尽可能少的 slot。思路: 按字节大小分类:

大小	数量	总字节
uint256 (32B)	14	448
address (20B)	5	100
uint128 (16B)	4	64
uint64 (8B)	4	32
uint32 (4B)	4	16
uint16 (2B)	5	10
uint8 (1B)	5	5
bool (1B)	5	5
bytes32 (32B)	4	128

总字节 808, 理论最少 $\text{ceil}(808/32)=26$ slot(packing 不能跨 slot, 现实下限稍高)。最优(约 26-28 slot):

```
contract GoodOrder {
    // slot 0-13: 14 个 uint256 各占一 slot
    uint256 b1; uint256 b2; ... uint256 b14;

    // slot 14-17: 4 个 bytes32
    bytes32 k1; bytes32 k2; bytes32 k3; bytes32 k4;

    // slot 18: address(20) + uint64(8) + uint32(4) = 32B
    address g1; uint64 h1; uint32 j1;

    // slot 19: address(20) + uint64(8) + uint32(4) = 32B
    address g2; uint64 h2; uint32 j2;

    // slot 20: address(20) + uint64(8) + uint32(4) = 32B
    address g3; uint64 h3; uint32 j3;

    // slot 21: address(20) + uint64(8) + uint32(4) = 32B
    address g4; uint64 h4; uint32 j4;

    // slot 22: address(20) + uint16(2)*5 + uint8(1)*2 = 32B
    address g5; uint16 c1; uint16 c2; uint16 c3; uint16 c4; uint16 c5; uint8 a1; uint8 a2;

    // slot 23: uint128(16) + uint128(16) = 32B
    uint128 i1; uint128 i2;

    // slot 24: uint128 + uint128
    uint128 i3; uint128 i4;

    // slot 25: 5 bool + 3 uint8 = 8 字节, 可后续追加更多小字段
    bool f1; bool f2; bool f3; bool f4; bool f5; uint8 a3; uint8 a4; uint8 a5;
}
```

从 50 slot 减到 ~26 slot, 部署省 $24 \times 22100 \approx 530k$ gas, 一次性写省等量 gas。

习题 3(基础): 修复重入

亲手写一次攻击合约比读 100 篇 The DAO 复盘都管用。这题让你站在攻击者视角, 用 `receive()` 递归把一个看起来人畜无害的 bank 抽干, 然后用三种方式修——CEI、OZ `ReentrancyGuard`、`ReentrancyGuardTransient`——并用 invariant 测试证明修复有效。

见 `exercises/03-reentrancy-fix/`, 写攻击合约 + 三种修复 + 不变量测试。攻击合约:

```
contract Attacker {
    VulnerableBank bank;
    constructor(VulnerableBank b) payable { bank = b; }

    function attack() external payable {
        bank.deposit{value: msg.value}();
        bank.withdraw();
    }

    receive() external payable {
        if (address(bank).balance ≥ 1 ether) bank.withdraw();
    }
}
```

修复 1(CEI): 先 `balances[msg.sender] = 0`, 再 `call`。修复 2: OZ `ReentrancyGuard`

```
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract SafeBank is ReentrancyGuard {
    function withdraw() external nonReentrant {
        uint256 bal = balances[msg.sender];
        (bool ok,) = msg.sender.call{value: bal}("");
        balances[msg.sender] = 0;
        require(ok);
    }
}
```

修复 3: `ReentrancyGuardTransient` (用法同上, 只改 import)。

Gas 对比(实测):

实现	withdraw gas
漏洞版(无防护)	30,123
CEI 版	30,221
<code>ReentrancyGuard</code>	35,341(多 ~5k)
<code>ReentrancyGuardTransient</code>	30,521(多 ~300)

不变量证明:

```

contract BankInvariantTest is StdInvariant, Test {
    SafeBank bank;
    BankHandler handler;

    function setUp() public {
        bank = new SafeBank();
        handler = new BankHandler(bank);
        targetContract(address(handler));
    }

    function invariant_solvent() public view {
        assertGe(address(bank).balance, handler.ghost_totalDeposited() -
handler.ghost_totalWithdrawn());
    }
}

```

习题 4(中等): EIP-712 签名 minter

NFT 合约, 用 owner 签名的 voucher mint(OZ EIP712.sol + ECDSA.sol, nonce 防重放, deadline 防过期)。骨架:

```

contract VoucherNFT is ERC721, EIP712 {
    using ECDSA for bytes32;

    bytes32 public constant VOUCHER_TYPEHASH = keccak256(
        "MintVoucher(address to,uint256 tokenId,uint256 nonce,uint256 deadline)"
    );

    address public immutable signer;
    mapping(uint256 => bool) public usedNonce;

    error VoucherExpired();
    error VoucherUsed();
    error InvalidSignature();

    constructor(address signer_)
        ERC721("VoucherNFT", "VNFT")
        EIP712("VoucherNFT", "1")
    { signer = signer_; }

    function mintWithVoucher(
        address to, uint256 tokenId, uint256 nonce, uint256 deadline,
        bytes calldata signature
    ) external {
        if (block.timestamp > deadline) revert VoucherExpired();
        if (usedNonce[nonce]) revert VoucherUsed();

        bytes32 structHash = keccak256(abi.encode(
            VOUCHER_TYPEHASH, to, tokenId, nonce, deadline
        ));
        bytes32 digest = _hashTypedDataV4(structHash);
        if (digest.recover(signature) != signer) revert InvalidSignature();

        usedNonce[nonce] = true;
        _mint(to, tokenId);
    }
}

```

习题 5(中等): UUPS 升级安全题

这道题在 RareSkills 的 audit bootcamp 上被反复使用——它在 30 行代码里同时藏了 5 个真实出过事故的坑。请你扮演审计师角色：

下面的升级有什么问题？

```
// V1
contract TokenV1 is UUPSUpgradeable, Initializable {
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    function initialize() external initializer { __UUPSUpgradeable_init(); }
    function _authorizeUpgrade(address) internal override {} // ← (a)
}

// V2
contract TokenV2 is UUPSUpgradeable, Initializable {
    address public newOwner; // ← (b)
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    function _authorizeUpgrade(address) internal override {}
}
```

5 个问题：① `_authorizeUpgrade` 无权限检查(任何人可升级)→ 加 `onlyOwner`；② V2 在前面插入 `newOwner` (`totalSupply` 从 slot 0 错位到 slot 1)→ 末尾追加或用 ERC-7201；③ V1 没初始化 `owner`，无 access control；④ 无 `_disableInitializers()`，实现合约可被外部 `initialize`；⑤ 无 `__gap`，父合约加字段会破坏布局。

正确 V1:

```
contract TokenV1 is UUPSUpgradeable, Initializable, OwnableUpgradeable {
    uint256 public totalSupply;
    mapping(address => uint256) public balanceOf;
    uint256[48] private __gap;

    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor() { _disableInitializers(); }

    function initialize(address owner) external initializer {
        __UUPSUpgradeable_init();
        __Ownable_init(owner);
    }

    function _authorizeUpgrade(address) internal override onlyOwner {}
}
```

习题 6(高级): Solady vs OZ SafeTransfer benchmark

对比三场景 gas: 标准 ERC20、USDT-like(不返回 bool)、失败 token(revert)。核心：

```

import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {SafeTransferLib} from "solady/utils/SafeTransferLib.sol";

contract Bench is Test {
    using SafeERC20 for IERC20;

    function test_gas_OZ() public {
        uint256 g = gasleft();
        IERC20(token).safeTransfer(to, 100);
        emit log_named_uint("OZ", g - gasleft());
    }

    function test_gas_Solady() public {
        uint256 g = gasleft();
        SafeTransferLib.safeTransfer(token, to, 100);
        emit log_named_uint("Solady", g - gasleft());
    }
}

```

预期:

场景	OZ SafeERC20	Solady SafeTransferLib	差异
标准 ERC20	~30,500	~22,200	-27%
USDT-like	~30,800	~22,100	-28%
revert	~25,000	~24,500	-2%(差别小, 都早 revert)

Solady 优势在 happy path: 免去 OZ `address.functionCall` 分配 memory 的开销。

CHAPTER 18

第 17 章 延伸阅读

读完这本模块只是起点——Solidity 生态每个月都有新 EIP、新审计案例、新工具。下面这份清单按"先看哪个最高 ROI"的顺序排: 官方文档保证准确性, 必读源码教你工业实践, 课程帮你系统补缺, 工具表是你日常工作流。每个链接都是我自己最近一年内查阅过的, 不是凑数的列表。

17.1 官方文档

- [Solidity 0.8.28 docs](#) (2024-10)
- [Foundry Book](#) (2026-04 仍持续更新)
- [OpenZeppelin Contracts v5](#)
- [Hardhat 3 docs](#) (2026-04 prod-ready beta)
- [EIP-1153 \(transient storage\)](#)
- [EIP-2612 \(permit\)](#)
- [EIP-712 \(typed signing\)](#)
- [EIP-1967 \(proxy storage\)](#)
- [EIP-7201 \(namespace storage\)](#)
- [EIP-7702 \(set code for EOA\)](#)
- [ERC-4626 \(vault\)](#)
- [EIP-2535 \(Diamond\)](#)
- [EIP-1167 \(Minimal Proxy / Clones\)](#)
- [EIP-3448 \(MetaProxy with immutable args\)](#)
- [EIP-5656 \(MCOPY\)](#)
- [EIP-2930 \(Optional access lists\)](#)
- [EIP-6093 \(Custom errors for ERC standards\)](#)
- [Solidity NatSpec Format](#)
- [Solidity SMTChecker](#)

17.2 备选语言/框架文档

- [Vyper 文档](#) (v0.4.3+)
- [Huff 文档](#)
- [Arbitrum Stylus 文档](#) (Rust + WASM)
- [Sway / Fuel 文档](#)
- [Solang 文档](#) (Solidity → Polkadot/Solana/Stellar)
- [Fe 项目](#) (实验中)
- [ApeWorX \(Ape\) 文档](#) (Brownie 继任者)
- [Permit2 文档](#)

17.3 课程

- [Cyfrin Updraft](#) (前 Patrick Collins / Chainlink Labs 团队, 免费)
- [Patrick Collins's Foundry Full Course](#) (YouTube 32 小时免费课)

- [RareSkills](#) (付费, 深度审计向)
- [Secureum](#) (合约安全速成)
- [LearnWeb3](#) (前端→合约转型路径)

17.4 必读源码

仓库	学什么
OpenZeppelin Contracts	标准合约的工业实现
Solady	极致 gas 优化
Solmate	gas 优化思路的源头
Uniswap V2 Pair	最小完整 AMM(本模块 14.6 详解)
Uniswap V4	hooks + transient storage 的极致案例
Morpho Blue	最简 lending market 设计
Pendle V2	复杂金融产品的工程化
EigenLayer	restaking + AVS 架构
Permit2	跨协议授权基础设施

17.5 工具与静态分析

- [Slither](#) (Crytic / Trail of Bits 静态分析, 事实标准)
- [Aderyn](#) (Cyfrin Rust 静态分析, 2026 主流)
- [Halmos](#) (a16z 符号执行 / fuzz-as-spec)
- [Echidna](#) (Haskell property fuzz, 与 Foundry invariant 互补)
- [Mythril](#) (符号执行, 老牌但慢)
- [Certora Prover](#) (商业形式化, Aave/Compound/Uniswap 都用)
- [horsefacts/weth-invariant-testing](#) (不变量测试经典实战范本)
- [Cyfrin/fuzz-testing-on-foundry](#) (fuzz / invariant 教学示例)

17.6 常见坑

- `address(this).balance` 不是内部账本 (selfdestruct 转入会超出记账)
- `tx.origin` 永远不用于鉴权 (钓鱼合约绕过)
- `block.timestamp` 不是随机源 (±15 秒可控)
- `for` 循环里 `external call` (gas 不可控 + 无长度限制 = DoS)
- ERC-20 fee-on-transfer (PAXG/SAFEMOON: `transferFrom` 100 对方收到 99)
- ERC-777 hook (绕过 reentrancy 防护)
- `call` 不限 gas (被调方可烧光 gas)
- `block.coinbase` PoS 后是 fee recipient, 不是 builder
- Proxy 实现合约未 `_disableInitializers()` (可被外部初始化)

- **storage gap** 不能跨继承层级：每层独立留 gap
 - **constant 数组**：`bytes32[] constant F00 = ...` (0.8.x 起允许，但每次访问按字节读 bytecode，gas 不便宜)
-